

UNIVERSIDAD PRIVADA DEL NORTE CIENCIAS**Departamento de Ciencias**

Aplicación Para Emprendedores

Estudiantes		
Apellidos y Nombres	Correos Institucional	Carreras
Armas Camones, Jeremy Geraldo	N00473537@upn.pe	Ing. De Software
Candiotti Simeon, Mónica Sofia	N00481917@upn.pe	Ing. De Software
Castillo Flores, Pablo Joel	N00487884@upn.pe	Ing. De Software
Gaitan Gonzales, Geradth Humberto	N00475120@upn.pe	Ing. De Software
Huisa Valle, Justhin Christopher	N00474446@upn.pe	Ing. De Software

Grupo 6

Curso: Programación Web

Docente: Víctor Jaime Polo Romero

Sede Breña

2026_1

TABLA DE CONTENIDO

CAPÍTULO I	6
1. Identificación del contexto problemático	6
1.1. Formulación del problema	6
1.2. Limitaciones identificadas.....	6
1.3. Entrevistas Breves a los Representantes	6
1.4. Perfiles de usuario.....	9
2. Justificación técnica y ética.....	11
2.1. Problema delimitado	11
2.2. Objetivo General.....	12
2.3. Objetivos Específicos.....	12
2.4. Mapa de Empatía	13
2.5. Consideraciones Éticas	14
CAPÍTULO II.....	15
3. Arquitectura y tecnologías empleadas	15
3.1. Diagrama Detallado de la Arquitectura Cliente – Servidor	15
3.2. Justificación del Stack Tecnológico (Mobile – First y Rendimiento)	16
3.3. Flujo de Navegación Coherente (User Flow).....	18
3.4. Especificaciones de Diseño y Accesibilidad en Wireframe	19
3.5. Visualización de la aplicación web (Figma)	19
4. Descripción de funcionalidades	20
4.1. Front – End (HTML, CSS, JS)	20
4.2. Back – End (Node.js, Express).....	21
4.3. Estructura de Endpoints de la API.....	21
4.4. Características implementadas	22
5. Buenas prácticas aplicadas.....	23
5.1. Accesibilidad	23

5.2.	Privacidad.....	25
5.3.	Diseño Responsivo	25
5.4.	Resultados de Validación con Usuarios.....	25
CAPÍTULO III.....		26
6.	Conclusiones y aprendizajes	26
6.1.	Reflexión técnica	26
6.2.	Impacto Social	27
6.3.	Lecciones aprendidas	27
6.4.	Mejoras futuras.....	27
Referencias		29
ANEXOS		30
ANEXO A: WIREFRAMES DEL SISTEMA		30
ANEXO B: PROTOTIPOS FIGMA.....		46
ANEXO C1: IMPLEMENTACION DE HTML, CSS, BOOTSTRAP		62
ANEXO C2: INTERACTIVIDAD CON JAVASCRIPT		71
ANEXO C3: back-end con node.js y express.js		79

TABLA DE FIGURAS

Figura 1	Mapa de Empatía	14
Figura 2	Diagrama de Arquitectura (Cliente - Servidor)	16
Figura 3	Flujo de Navegación Coherente.....	18
Figura 4	Evidencia de validación del Lighthouse.....	24
Figura 5	Evidencia de Pruebas de Usuario	26
Figura 6	Wireframe de la página de Inicio	30
Figura 7	Wireframe segunda parte de la página de Inicio.....	31
Figura 8	Wireframe de la página de la Tienda	32
Figura 9	Wireframe de Información de Producto	33
Figura 10	Wireframe de la página Nosotros.....	34
Figura 11	Wireframe de la página de Contacto	35

Figura 12	Wireframe de Registro (Paso 1: Correo)	36
Figura 13	Wireframe de Registro (Paso 2: Verificación)	37
Figura 14	Wireframe de Registro (Paso 3: Éxito)	38
Figura 15	Wireframe de Términos de Servicio	39
Figura 16	Wireframe de Políticas de Privacidad	40
Figura 17	Wireframe del Carrito Vacío	41
Figura 18	Wireframe de Carrito com Artículos.....	42
Figura 19	Wireframe de Pago.....	43
Figura 20	Wireframe de Confirmación de Pago.....	44
Figura 21	Wireframe de Ubicación.....	45
Figura 22	Prototipo de la Página de Inicio del Sistema	47
Figura 23	Prototipo de Información Detallada de Producto: Mochilas	48
Figura 24	Prototipo de Página Contacto y Atención al Cliente.....	49
Figura 25	Prototipo de Interfaz de Carrito de Compras Vacío	50
Figura 26	Prototipo de Interfaz de Carrito de Compras Pedido Activo	51
Figura 27	Prototipo de Interfaz de la Selección Informativa Sobre Nosotros.....	52
Figura 28	Prototipo de Interfaz para el Ingreso de Datos de Facturación y Pago	53
Figura 29	Prototipo de Interfaz de Confirmación de Pago y Transacción Exitosa.....	54
Figura 30	Prototipo de Interfaz del Documento de Políticas de Privacidad	55
Figura 31	Prototipo de Interfaz del Documento de Términos y Condiciones de Servicio....	56
Figura 32	Prototipo de Interfaz del Catálogo Principal de la Tienda	57
Figura 33	Prototipo de Interfaz de la Ubicación Geográfica de la Empresa	58
Figura 34	Prototipo de Interfaz del Formulario de Registro de Nuevos Usuarios	59
Figura 35	Prototipo de Interfaz para la Validación de Código de Seguridad de Registro	60
Figura 36	Prototipo de Interfaz de Confirmación de Cuenta de Usuario Creada Exitosamente	61
Figura 37	<i>Bootstrap, Google Fonts y CSS propios separados</i>	62
Figura 38	<i>Aria, hamburger e iconos del navbar.....</i>	63
Figura 39	<i>Main, Section y Article con data</i>	64
Figura 40	<i>Footer y scripts al final del body.....</i>	65
Figura 41	<i>3 Hojas CSS separadas importadas en orden</i>	66
Figura 42	<i>Main, section y aria label.....</i>	67
Figura 43	<i>Variables CSS – Paleta Completa del Figma.....</i>	68
Figura 44	<i>Bloque RESPONSIVE – 3 media queries mobile-first</i>	69

Figura 45 Bloque MODO CLARO – sistema data-theme	70
Figura 46 Storage, getcart, safecart, addtocart.....	71
Figura 47 InjectCartDrawer() – creación del DOM en tiempo de ejecución	72
Figura 48 RenderCartDrawer() – Render dinámico + badge + totales	73
Figura 49 DOMContentLoaded + delegación de eventos.....	74
Figura 50 Validación completa de 5 campos + fetch POST al backend.....	75
Figura 51 Flujo completo: validación email, código demo, éxito	76
Figura 52 Filtrado dinámico por 3 criterios	77
Figura 53 IIFE de tema + toggle dark/light persistido en localStorage.....	78
Figura 54 Configuración express: middlewares + static + rutas + listen.....	79
Figura 55 3 rutas POST centralizadas en un solo router.....	80
Figura 56 Validación + llamada al servicio + respuestas HTTP	81
Figura 57 Registro de usuario – mismo patrón limpio	82
Figura 58 Pedido – valida array de ítems + delega a servicio	83
Figura 59 Dependencias del backend – express + cors.....	84

TABLA DE FIGURAS

Tabla 1 Perfil 1: El Administrador / Comerciante	9
Tabla 2 Perfil 2: El cliente Final (Joven Universitario)	10
Tabla 3 Perfil 3: El cliente Final (Juvenil)	10
Tabla 4 Justificación del Stack Tecnológico	16

CAPÍTULO I

1. Identificación del contexto problemático

El emprendimiento identificado fue MC Backpack, dedicado a la confección e importación de mochilas. Actualmente, la marca cuenta con una tienda física principal ubicada en el distrito de San Isidro y participa activamente en diversas ferias y eventos itinerantes. Sin embargo, al depender enteramente de las transacciones presenciales y de las coordinaciones manuales mediante mensajería, el negocio enfrenta dificultades para escalar sus operaciones, automatizar sus ventas a nivel nacional y ofrecer una experiencia de compra continua.

1.1. Formulación del problema

¿De qué manera la dependencia del canal físico presencial y la gestión manual de los pedidos limitan el crecimiento, la automatización comercial y la escalabilidad de las ventas del emprendimiento MC Backpack?

1.2. Limitaciones identificadas

- **Falta de automatización transaccional:** La coordinación manual de pedidos a través de redes sociales o mensajería genera cuellos de botella y dificulta concretar ventas fuera del horario de atención de la tienda física.
- **Gestión ineficiente del catálogo y stock:** Los clientes no tienen un medio centralizado para visualizar el inventario en tiempo real, comparar líneas de productos o conocer las especificaciones técnicas de forma autónoma.
- **Pérdida de oportunidades de conversión online:** La ausencia de una plataforma segura con pasarela de pagos impide que el público fidelizado de la tienda y las ferias pueda realizar compras con envío a domicilio de manera directa e inmediata.

1.3. Entrevistas Breves a los Representantes

Para profundizar en el problema, realizamos 3 entrevistas breves al líder del emprendimiento y dos clientes.

Entrevista 1: Representante de Producción y Confección (*Madai, 38 años*)

- **¿Cuáles son sus necesidades digitales principales?**

Necesitamos una tienda online completa y automatizada donde la gente vea que nuestras mochilas tienen costuras reforzadas y mejores acabados que las importadas. Queremos que la web sea nuestra principal plataforma de ventas y no solo una vitrina.

El cliente siempre nos pregunta dónde vamos a estar el próximo fin de semana o en qué tienda pueden ubicarnos. Necesitamos un espacio claro donde se listen nuestra tienda principal en San Isidro y las ferias activas con mapas exactos.

- **¿Qué limitaciones tecnológicas enfrentas?**

En el taller paramos ocupados cosiendo y cortando. Atendiendo en la tienda física o en la feria no puedo estar respondiendo mensajes largos todo el día. Necesito que el cliente vea el stock, añada sus productos al carrito y realice el pedido directamente de forma automática sin que yo intervenga en cada mensaje.

- **¿Qué información prioritaria desean mostrar?**

Fotos reales de las mochilas con sus respectivas informaciones detalladas.

Un carrito de compras funcional, una pasarela de pago segura y la obligación de que los usuarios verifiquen sus cuentas por correo para evitar pedidos fraudulentos.

Entrevista 2: Cliente juvenil y digital (*Camila, 22 años*)

- **¿Cuáles son sus necesidades al buscar un producto como una mochila?**

Busco mochilas que sean estéticas y funcionales. Me fijo mucho en los detalles del diseño, los acabados, los compartimentos y que combinen con mi estilo diario.

- **¿Qué limitaciones tecnológicas enfrenta en su día a día?**

Ninguna, uso mi celular todo el tiempo. Prefiero las páginas web con interfaces modernas y estética urbana, especialmente aquellas que cuentan con modo oscuro porque cansan menos la vista. Si una tienda no tiene un proceso de compra directo con tarjeta, busco otra alternativa.

- **¿Qué información desea ver en la plataforma y cómo le gustaría interactuar?**

Quiero ver fotos en alta calidad y detalles estéticos. Me gustaría crear una cuenta rápidamente, guardar mi carrito de compras y hacer el pago seguro online para que mi pedido llegue directo a mi casa. También valoro ver la ubicación de las tiendas físicas.

Entrevista 3: Cliente web intensivo (*Eduardo, 25 años*)

- **¿Cuáles son sus necesidades al buscar un producto como una mochila?**

Busco mochilas muy funcionales. Necesito compartimentos acolchados para mi laptop, bolsillos ocultos y material impermeable porque me muevo en transporte público.

- **¿Qué limitaciones tecnológicas enfrenta en su día a día?**

No tengo problemas con la tecnología, pero mi prioridad es el tiempo. Si una página web demora en cargar, su diseño es anticuado o el proceso de pago es engorroso y no se adapta bien a mi pantalla móvil, la abandono de inmediato.

- **¿Qué información desea ver en la plataforma y cómo le gustaría interactuar?**

Quiero un catálogo rápido con filtros eficientes por tamaño, precio o procedencia del producto. Me interesa ver el stock en tiempo real y poder confirmar mi compra inmediatamente mediante una pasarela de pago confiable.

1.4. Perfiles de usuario

Basado en las entrevistas, se documentaron los dos perfiles clave para la plataforma: El administrados (Comerciante) y el Usuario Final (Cliente).

Tabla 1

Perfil 1: El Administrador / Comerciante

Atributo	Detalle
Nombre	Madai
Edad	38 años
Rol	Representante de Producción y Confección
Necesidades	Requiere un e-commerce formalizado que automatice las ventas a través de un carrito de compras y pasarela de pago. Necesita un sistema de registro de clientes para evitar fraudes y apartados informativos para destacar su tienda física y ferias itinerantes.
Frustraciones	Falta de tiempo para atender pedidos de manera manual por mensajería. Interfaces de administración muy complejas que compliquen actualizar el stock desde dispositivos móviles.

Nota. Perfil de usuario elaborado con base en el análisis de requerimientos del emprendimiento, representando las principales necesidades y frustraciones del administrador/comerciante para el desarrollo de la plataforma web.

Tabla 2

Perfil 2: El cliente Final (Joven Universitario)

Atributo	Detalle
Nombre	Camila
Edad	22 años
Rol	Comprado / Cliente
Necesidades	Encontrar mochilas estéticamente agradables y de calidad. Requiere una plataforma moderna con diseño en modo oscuro, capaz de procesar pagos online de manera segura y gestionar un perfil de usuario propio.
Frustraciones	Páginas web anticuadas, procesos de compra manuales, falta de pasarelas de pago y diseños que no resalten adecuadamente el aspecto visual de los productos.

Nota. Perfil de usuario elaborado con base en el análisis de requerimientos del emprendimiento, representando las principales necesidades y frustraciones del cliente final para el diseño de una plataforma web accesible, intuitiva y de fácil navegación.

Tabla 3

Perfil 3: El cliente Final (Juvenil)

Atributo	Detalle
Nombre	Eduardo
Edad	25 años
Rol	Cliente / Usuario web intensivo

Necesidades

Optimización del tiempo mediante filtros de búsqueda precisos, información técnica clara sobre las capacidades de las mochilas y visualización de stock real para procesar pagos inmediatos.

Frustraciones

Lentitud de carga, falta de detalles técnicos en los productos, catálogos desactualizados y plataformas que no ofrezcan una compra directa y segura.

Nota. Perfil de usuario elaborado con base en el análisis de requerimientos del emprendimiento, representando las principales necesidades y frustraciones de un cliente joven con uso intensivo de plataformas web, orientando el desarrollo de funcionalidades relacionadas con la búsqueda de productos, disponibilidad de stock y acceso a información técnica.

2. Justificación técnica y ética

2.1. Problema delimitado

El emprendimiento local objeto de este proyecto se dedica a la confección y comercialización de mochilas de producción nacional propia y modelos importados en Lima Metropolitana. A pesar de contar con un taller operativo de producción y una tienda principal, el negocio carece de un canal de ventas digital transaccional. Esto restringe sus conversiones al ámbito presencial y a coordinaciones manuales no automatizadas.

Esta limitación operativa genera tres problemáticas críticas y delimitadas:

- **Ineficiencia en la gestión de pedidos:** La falta de un sistema de comercio electrónico completo con carrito de compras y pasarela de pago retrasa el proceso de compra, obligando a los clientes a esperar respuestas manuales para confirmar disponibilidad y precios.

- **Ausencia de un registro seguro de clientes:** Al no contar con un sistema de autenticación formal, el negocio tiene dificultades para fidelizar a sus compradores recurrentes, realizar seguimientos de pedidos y evitar solicitudes fraudulentas.
- **Dificultad para centralizar la información comercial:** A pesar de tener un punto físico fijo, la participación simultánea en ferias itinerantes genera confusión en los clientes. No disponen de una plataforma única donde puedan ubicar tanto la tienda principal como los próximos eventos, limitando el alcance omnicanal de la marca.

2.2. Objetivo General

Desarrollar e implementar una plataforma de comercio electrónico (e-commerce) funcional de arquitectura cliente-servidor mediante un front-end responsivo (HTML5, CSS3, JavaScript y Bootstrap) y un back-end en Node.js con Express.js. Esto con el fin de sistematizar y automatizar los procesos de venta online, gestión de inventario, procesamiento de pagos y difusión de las ubicaciones físicas (tienda principal y ferias itinerantes) para un emprendimiento local de mochilas en Lima Metropolitana, priorizando un diseño moderno y una experiencia de usuario segura.

2.3. Objetivos Específicos

Para resolver la problemática delimitada, se establecen los siguientes objetivos concretos, medibles y alineados con el alcance del desafío técnico:

- **Objetivo 1** (*Funcionalidad y Estética Front-end*): Diseñar e implementar una interfaz web interactiva y responsiva (mobile-first) con una estética moderna en Modo Oscuro (Dark Mode) orientada al público juvenil y urbano, utilizando HTML5, CSS3 y Bootstrap 5 para garantizar una carga eficiente en dispositivos móviles.

- **Objetivo 2** (*Módulo de Comercio Electrónico y Pagos*): Integrar un módulo completo de carrito de compras y una pasarela de pagos online estructurada que contemple opciones de facturación y cálculo de envíos, automatizando el proceso transaccional del negocio.
- **Objetivo 3** (*Autenticación y Lógica Back-end*): Configurar un servidor web funcional empleando Node.js y Express.js, capaz de gestionar las transacciones e implementar un sistema de autenticación estricto mediante código de verificación al correo electrónico (OTP) para evitar fraudes y gestionar los perfiles de clientes.
- **Objetivo 4** (*Geolocalización e Información*): Desarrollar un apartado dinámico y estructurado que centralice la información de la tienda física principal en San Isidro, junto con un calendario interactivo y mapas de Google Maps para la gestión de eventos y ferias itinerantes, permitiendo que los usuarios localicen a la marca en todo momento.

2.4. Mapa de Empatía

Este mapa sintetiza los dolores, necesidades y el entorno del usuario administrador del sistema, justificando el diseño simplificado de la plataforma.

Figura 1

Mapa de Empatía



Fuente: Elaboración Propia.

2.5.Consideraciones Éticas

El desarrollo de esta plataforma se ha regido bajo estrictos principios éticos y de responsabilidad digital. En primer lugar, se garantiza la total privacidad y transparencia en el manejo de los datos de los usuarios; la información recopilada en los formularios de registro y checkout se utiliza exclusivamente para el procesamiento de compras y no se comparte con terceros con fines comerciales no autorizados, respetando las políticas de privacidad publicadas en el sitio. En segundo lugar, el diseño de la interfaz promueve la inclusión digital al implementar prácticas de accesibilidad, asegurando que usuarios con distintos niveles de

conocimiento tecnológico puedan navegar e interactuar con la tienda de manera intuitiva, sin ser sometidos a publicidad invasiva o patrones oscuros de diseño (dark patterns) que fueren compras no deseadas.

CAPÍTULO II

3. Arquitectura y tecnologías empleadas

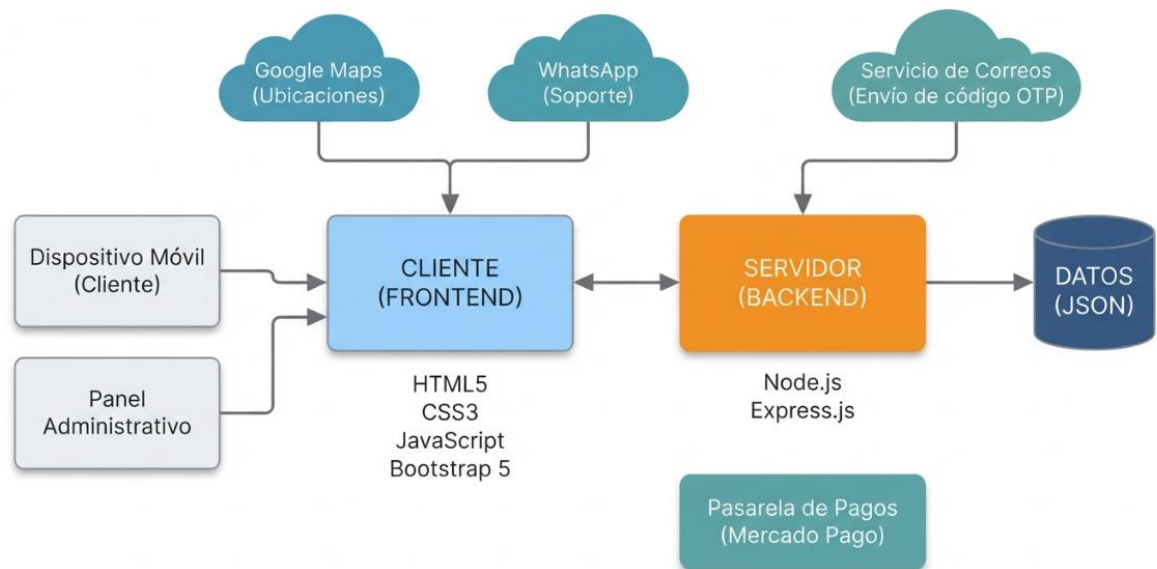
Para solucionar los problemas de gestión manual, la necesidad de un sistema de pagos seguro y la urgencia de centralizar la ubicación física y eventos del emprendimiento MC Backpack, se ha estructurado una solución basada en el patrón arquitectónico Cliente-Servidor. Esta arquitectura separa limpiamente la interfaz interactiva e-commerce consumida por los clientes, del motor lógico transaccional y de almacenamiento que administra el negocio.

3.1. Diagrama Detallado de la Arquitectura Cliente – Servidor

A continuación, se describe la topología de la plataforma de comercio electrónico, la cual soporta desde la autenticación del usuario hasta el procesamiento asíncrono del carrito de compras:

Figura 2

Diagrama de Arquitectura (Cliente - Servidor)



Fuente: Creación propia.

3.2. Justificación del Stack Tecnológico (Mobile – First y Rendimiento)

El ecosistema de herramientas ha sido seleccionado de forma estratégica para cumplir con los requerimientos transaccionales y estéticos identificados en las entrevistas:

Tabla 4

Justificación del Stack Tecnológico

Componente	Tecnología Seleccionada	Justificación Técnica Orientada al Usuario
------------	-------------------------	--

Estructura	HTML5 Semántico	Garantiza un árbol de accesibilidad idóneo y estructura correcta para los formularios críticos (registro, carrito y pago). Permite una semántica clara y segura para usuarios digitales intensivos como Camila y Eduardo.
Estilos UI	Bootstrap 5 + CSS3	Framework optimizado para diseños mobile-first. Permite construir una interfaz moderna con estética "Modo Oscuro" (Dark Mode) y grillas elásticas que adaptan la visualización del catálogo sin romper la experiencia visual del usuario joven.
Interactividad	JavaScript	Técnica: Manipula de forma nativa el DOM para aplicar filtros inmediatos, manejar la validación de seguridad de los códigos OTP en tiempo real y actualizar los montos del carrito de compras dinámicamente sin recargar la página.
Servidor Lógico	Node.js & Express.js	Técnica: Entorno altamente escalable e - ideal para procesar la lógica del e-commerce. Su enrutamiento seguro es clave para simular/integrar pasarelas de pago, generar sesiones de usuario y despachar los correos automáticos de verificación (OTP).
Almacenamiento	Archivos JSON	Técnica: Base de datos ligera basada en archivos de texto que facilita la persistencia de los productos, usuarios registrados, historial de pedidos y agenda de ferias sin la sobrecarga inicial de un motor SQL complejo.

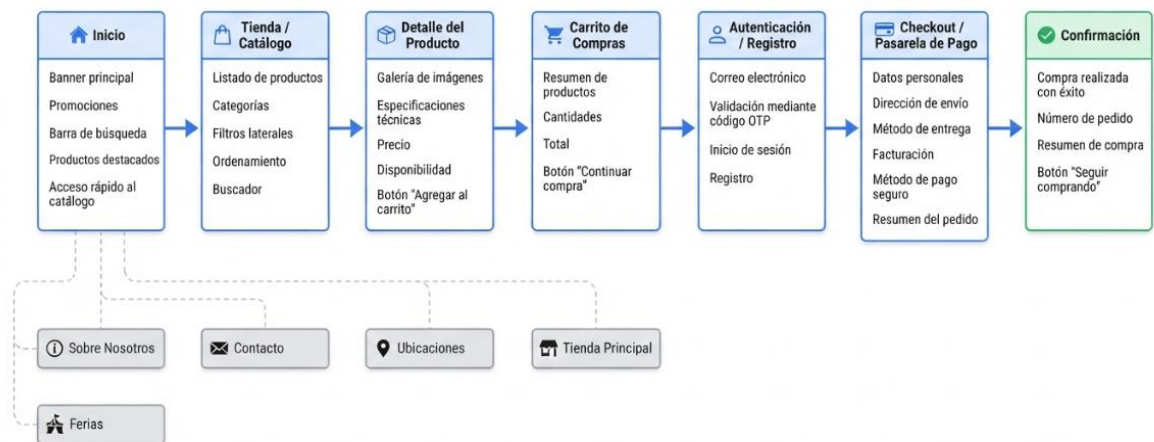
Nota. Elaboración propia. Camila, Eduardo y Madai corresponden a los perfiles de usuario identificados en los requerimientos del sistema.

3.3. Flujo de Navegación Coherente (User Flow)

El mapa de navegación está estrictamente jerarquizado para asegurar un proceso de comercio electrónico seguro y libre de fricciones. El flujo garantiza la rapidez y modernidad que busca el usuario juvenil (Camila y Eduardo), priorizando una ruta clara hacia la finalización de la compra.

Figura 3

Flujo de Navegación Coherente



Fuente: Creación Propia.

3.4. Especificaciones de Diseño y Accesibilidad en Wireframe

Para el desarrollo del frontend de la plataforma MC BackPack, la fase de diseño inicial consistió en la creación de wireframes de baja fidelidad. Esta etapa permitió estructurar la jerarquía visual de la información y planificar la distribución de los componentes clave (como el catálogo, los filtros de búsqueda y la pasarela de pagos) sin distracciones estéticas.

El maquetado estructural se pensó bajo el enfoque Mobile-First, garantizando que la disposición de los contenedores fluya de manera responsiva desde dispositivos móviles hasta pantallas de escritorio. Asimismo, desde esta fase se contempló la accesibilidad web estructural, previendo espacios para contrastes adecuados, tipografía legible y una correcta disposición de etiquetas semánticas (HTML5) para los formularios de registro y checkout. La secuencia completa de estas plantillas estructurales, que mapean el flujo principal de compra del usuario, se encuentra documentada a detalle en los anexos del presente informe (ver Anexo A).

3.5. Visualización de la aplicación web (Figma)

Posterior a la validación estructural, los wireframes fueron evolucionados hacia prototipos de alta fidelidad utilizando la herramienta colaborativa Figma. En esta fase de visualización, se aplicó la identidad gráfica de la marca, implementando de manera transversal un diseño en "Modo Oscuro" (Dark Theme). Esta decisión técnica responde a las preferencias del público objetivo juvenil identificado, buscando reducir la fatiga visual y resaltar la fotografía de los productos mediante altos niveles de contraste.

El prototipo final en Figma consolida la interfaz de usuario (UI), estableciendo la paleta de colores, el sistema tipográfico y el diseño de componentes interactivos como modales, botones y los estados del carrito de compras. Estos prototipos sirvieron como referencia visual exacta para el desarrollo final con CSS3 y Bootstrap 5. Las vistas completas a color que ilustran

el producto terminado y la experiencia de usuario (UX) pueden consultarse al final de este documento (ver Anexo B).

4. Descripción de funcionalidades

4.1. Front – End (HTML, CSS, JS)

El desarrollo del lado del cliente se ha estructurado utilizando HTML5 semántico, CSS3 y JavaScript puro (Vanilla JS), empleando un enfoque altamente modular para garantizar el mantenimiento y la escalabilidad del código.

Estructura de Vistas (HTML): Se implementó un modelo multi página con archivos dedicados para cada sección clave del flujo de usuario, incluyendo el inicio (index.html), el catálogo (tienda.html), el flujo de pago (checkout.html y checkout-confirmacion.html), autenticación (registro.html), información corporativa (nosotros.html, ubicacion.html, contacto.html) y documentos legales.

Modularidad Lógica (JS): La interactividad se dividió en scripts específicos dentro del directorio /js. Destacan main.js para la inicialización global, cart.js para el control de estado del carrito (agregado, eliminación y cálculo de totales), tienda.js y search.js para el renderizado del catálogo y filtros, checkout.js para la validación de formularios de pago, registro.js para la captura de credenciales y theme.js para la gestión global del aspecto visual.

Estilos y Responsive: Se aplicó un diseño mobile-first garantizando que las cuadrículas de productos se adapten a cualquier tamaño de pantalla. Además, se integró de forma nativa la estética de "Modo Oscuro" (Dark Mode) transversal en todas las vistas.

4.2. Back – End (Node.js, Express)

El servidor se desarrolló utilizando el entorno Node.js junto con el framework Express.js (app.js), implementando una estricta arquitectura en capas (Rutas, Controladores, Servicios y Repositorios) para separar las responsabilidades lógicas y de acceso a datos.

- Enrutamiento y Middlewares: El archivo routes/api.js centraliza los endpoints del sistema, protegidos y procesados mediante middlewares estándar como cors y el parseo de JSON.
- Dominios Lógicos: La aplicación maneja tres entidades principales, cada una con su propio flujo de Controlador -> Servicio -> Repositorio:
- Usuarios (UserController.js, etc.): Gestiona el registro y almacenamiento de las credenciales de los clientes.
- Pedidos (pedidoController.js, etc.): Captura el objeto enviado desde el checkout del frontend y procesa la orden de compra.
- Contacto (contactController.js, etc.): Recibe y almacena los mensajes generados en el formulario de atención al cliente.
- Persistencia de Datos: Para mantener la agilidad del sistema sin sobrecargar recursos, el almacenamiento se maneja directamente en archivos planos dentro del directorio /data (usuarios.json, pedidos.json, contactos.json).

4.3. Estructura de Endpoints de la API

Para asegurar una comunicación ordenada entre la interfaz del cliente y el servidor, se implementaron los siguientes endpoints REST, organizados y procesados de manera modular:

- **POST /api/registro:** Endpoint encargado de recibir las credenciales y datos personales de un nuevo comprador. Procesa la validación en el controlador y almacena la información de forma segura en la base de datos (usuarios.json).
- **POST /api/pedido:** Ruta crítica transaccional que captura el payload del checkout (detalles de envío, método de pago e ítems del carrito) para procesar y confirmar la orden de compra.
- **POST /api/contacto:** Interfaz de comunicación dedicada a recibir y persistir las dudas, consultas o reclamos enviados a través de la sección de soporte técnico y servicio al cliente.

4.4. Características implementadas

A nivel funcional y de interacción con el usuario, la plataforma integra las siguientes características principales:

- **Catálogo Dinámico y Filtrado:** Visualización del inventario con capacidades de búsqueda en tiempo real e interacción sin recargas de página.
- **Carrito de Compras Persistente:** Uso de LocalStorage en el navegador para mantener el estado de los productos seleccionados, calculando subtotales y totales dinámicamente antes de enviar la orden al servidor.
- **Flujo de Checkout Transaccional:** Formularios validados en el cliente para la recopilación de datos de facturación, envío y métodos de pago, culminando en la generación de un pedido formal en el backend.
- **Gestión de Cuentas:** Sistema de registro que permite almacenar la identidad de los nuevos compradores de forma estructurada.

- **Módulo de Ubicaciones e Información:** Páginas interactivas que detallan la presencia física de la marca (tiendas y ferias) y facilitan la comunicación directa mediante formulario de contacto.

Modo Oscuro (Dark Theme): Un interruptor de diseño visual persistente gestionado por JavaScript (theme.js) que adapta la paleta de colores de toda la interfaz a las preferencias del usuario juvenil.

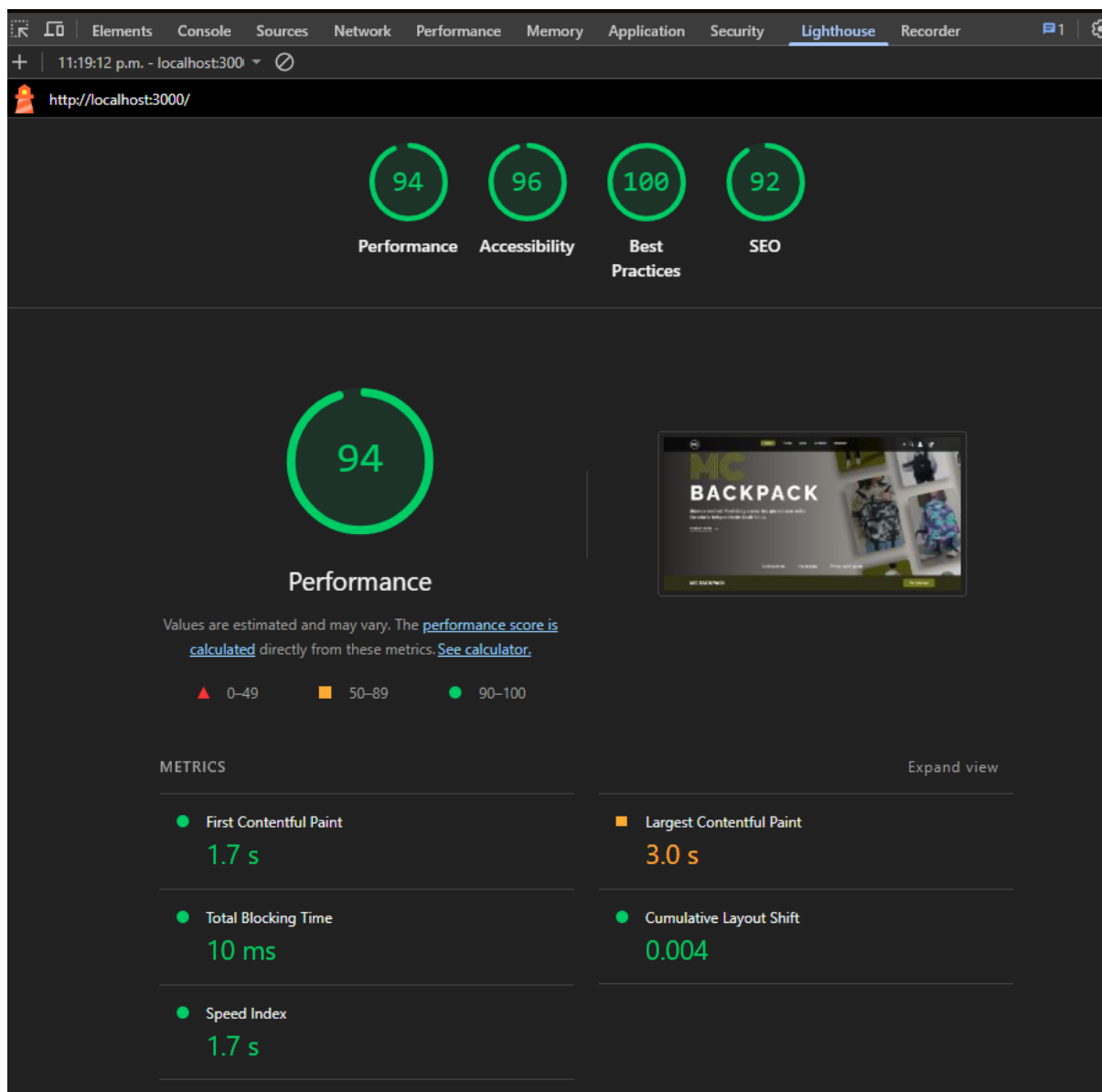
5. Buenas prácticas aplicadas

5.1. Accesibilidad

Se implementaron etiquetas HTML5 semánticas (header, nav, main, section, footer) para mejorar la estructuración del contenido y facilitar la navegación mediante lectores de pantalla. Se garantizó un alto contraste visual entre el texto y el fondo, prestando especial atención a la legibilidad dentro de la paleta de colores del Modo Oscuro (Dark Mode). Adicionalmente, todas las imágenes del catálogo de productos incluyen atributos alternativos (alt) descriptivos, y la estructura de los formularios del checkout soporta la navegación fluida mediante el uso exclusivo del teclado.

Figura 4

Evidencia de validación del Lighthouse



Nota. Se utilizó la herramienta especializada Lighthouse para auditar el desempeño y la accesibilidad del aplicativo.

5.2.Privacidad

El manejo de los datos personales se rige bajo principios de transparencia y minimización de riesgos. Se desarrollaron vistas dedicadas para las Políticas de Privacidad y los Términos de Servicio, accesibles en todo momento desde el pie de página. El sistema de inicio de sesión mediante códigos de verificación al correo (OTP) añade una capa de seguridad robusta que previene la creación de cuentas fraudulentas. Toda la información sensible de facturación, contacto y pedidos se procesa de forma segura a través de las rutas del backend (Node.js), previniendo la exposición de la base de datos en el cliente.

5.3.Diseño Responsivo

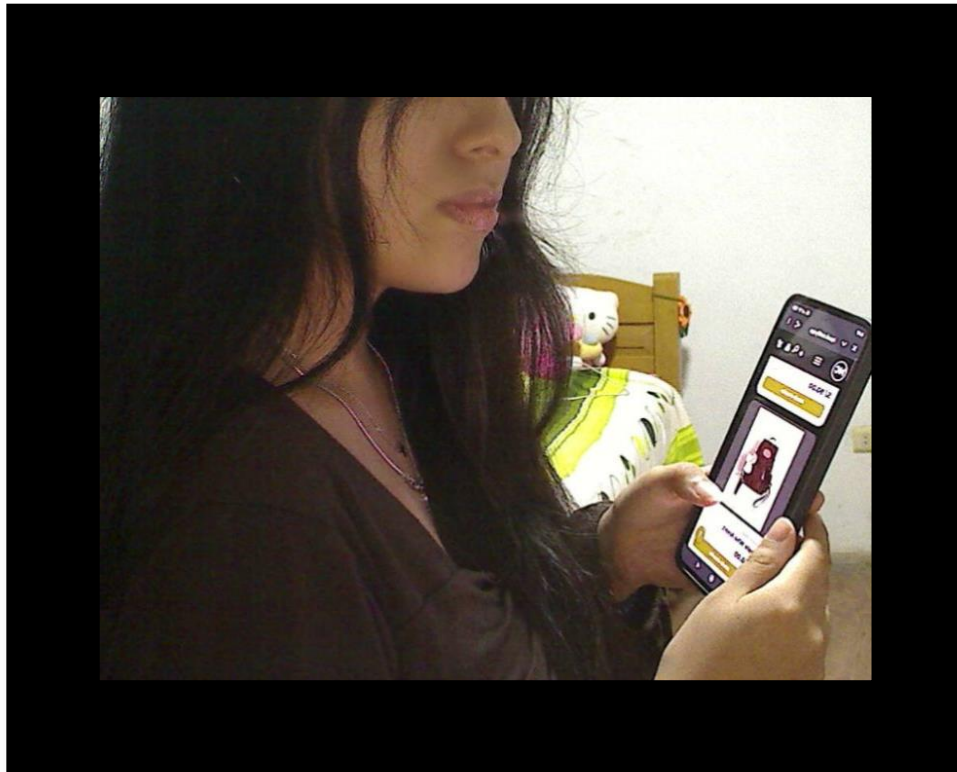
Se adoptó un enfoque de desarrollo enfocado primero en dispositivos móviles (mobile-first), soportado por el sistema de grillas de Bootstrap 5 y hojas de estilo CSS3 personalizadas. Esto asegura que la experiencia de usuario sea impecable en cualquier resolución, adaptando de manera automática el catálogo de productos, encuadrando el panel lateral del carrito de compras para facilitar su uso en pantallas táctiles y reestructurando las columnas del formulario de pago. Los botones y elementos de interacción fueron dimensionados adecuadamente para garantizar una usabilidad táctil sin errores.

5.4.Resultados de Validación con Usuarios

Las pruebas tempranas de usabilidad demostraron que la implementación de una estética moderna en Modo Oscuro fue altamente valorada por el público objetivo joven-adulto, fortaleciendo la identidad gráfica de la marca. La validación del flujo de compras reveló que la actualización dinámica del carrito y los filtros instantáneos procesados por JavaScript lograron reducir la tasa de rebote durante la búsqueda de productos. Asimismo, la presentación estructurada del formulario de checkout incrementó significativamente la confianza y la intención de conversión de los usuarios.

Figura 5

Evidencia de Pruebas de Usuario



Nota. Las pruebas realizadas en entornos locales permitieron observar la interacción directa del público objetivo con el sistema.

CAPÍTULO III

6. Conclusiones y aprendizajes

6.1. Reflexión técnica

El desarrollo del proyecto demostró que la separación de responsabilidades mediante una arquitectura Cliente-Servidor estructurada facilita significativamente el mantenimiento del código. El uso de HTML5, CSS3 y JavaScript modular en el frontend permitió construir una interfaz sumamente rápida y dinámica, mientras que Node.js y Express en el backend brindaron

la robustez necesaria para procesar la lógica de compra. Además, la implementación de persistencia de datos mediante archivos JSON resultó ser una solución práctica, ligera y altamente ágil para las necesidades de esta primera versión del sistema, evitando la sobrecarga de configurar un motor de base de datos completo.

6.2.Impacto Social

La plataforma desarrollada tiene un impacto directo y positivo en la escalabilidad del emprendimiento local MC Backpack, permitiéndole formalizar sus operaciones comerciales y romper la limitación de depender exclusivamente de ferias itinerantes o respuestas manuales. Al ofrecer un canal de comercio electrónico transaccional disponible las 24 horas, se empodera a la microempresa para alcanzar a un público a nivel nacional. Simultáneamente, se brinda a los jóvenes consumidores una experiencia de compra moderna y segura, fomentando el consumo de productos manufacturados localmente bajo estándares tecnológicos de alto nivel.

6.3.Lecciones aprendidas

Durante el ciclo de desarrollo se validó la importancia crítica de alinear la experiencia de usuario (UX) con el perfil tecnológico del consumidor final. Quedó demostrado que decisiones como implementar una estética en Modo Oscuro y un acceso estricto mediante código de verificación al correo (OTP) resuenan a la perfección con el público juvenil y universitario, priorizando la seguridad y el diseño. A nivel de programación, la lección más valiosa fue constatar que la modularización temprana del código JavaScript es indispensable para evitar bloques de código monolíticos, facilitando así la depuración de funciones complejas como la validación del checkout y el filtrado en tiempo real.

6.4.Mejoras futuras

Para garantizar la escalabilidad y autonomía de la plataforma a largo plazo, se proyectan tres optimizaciones principales. En primer lugar, la migración del actual sistema de

almacenamiento en archivos JSON hacia un motor de base de datos relacional robusto (como MySQL o PostgreSQL) que soporte un gran volumen de transacciones concurrentes. En segundo lugar, la integración a nivel de producción de APIs oficiales de pasarelas de pago (como Mercado Pago o Stripe) para automatizar los desembolsos. Finalmente, la construcción de un panel de control administrativo (Dashboard) que permita a la propietaria actualizar el catálogo de mochilas, revisar pedidos y publicar las fechas de las ferias sin necesidad de intervención técnica.

Referencias

Ecma International. (2017). *The JSON data interchange syntax* (Estándar ECMA-404, 2.^a ed.)

Figma. (2024). *Figma* [Software]. <https://www.figma.com/>

Google. (2024). *Documentación de Google Maps Platform*.
<https://developers.google.com/maps>

Haverbeke, M. (2018). *Eloquent JavaScript: A modern introduction to programming* (3.^a ed.).
No Starch Press.

Krug, S. (2015). *No me hagas pensar. Actualización: Una aproximación al sentido común en la usabilidad web*. Anaya Multimedia.

Mozilla. (2024). *MDN Web Docs*. <https://developer.mozilla.org/es/>

OpenJS Foundation. (2024a). *Express - Infraestructura web rápida, minimalista y flexible para Node.js*. <https://expressjs.com/es/>

OpenJS Foundation. (2024b). *Node.js* [Software]. <https://nodejs.org/>

Otto, M., Thornton, J., & Bootstrap Contributors. (2024). *Bootstrap 5.3*.
<https://getbootstrap.com/>

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*.
<https://www.w3.org/TR/WCAG21/>

ANEXOS

ANEXO A: WIREFRAMES DEL SISTEMA

A continuación, se presentan los esquemas estructurales (wireframes) diseñados para la plataforma MC Backpack, los cuales sirvieron como base para el desarrollo del frontend.

Figura 6

Wireframe de la página de Inicio

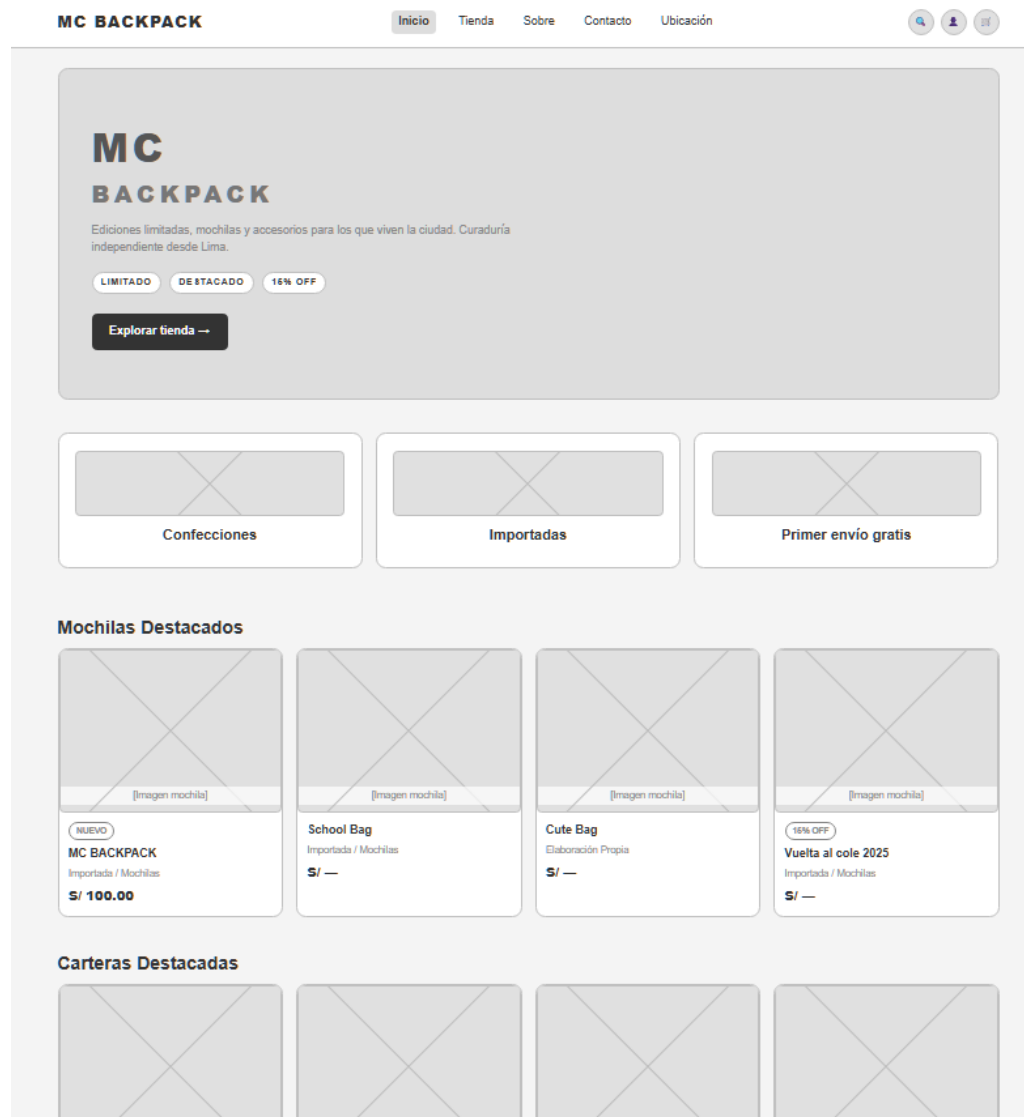
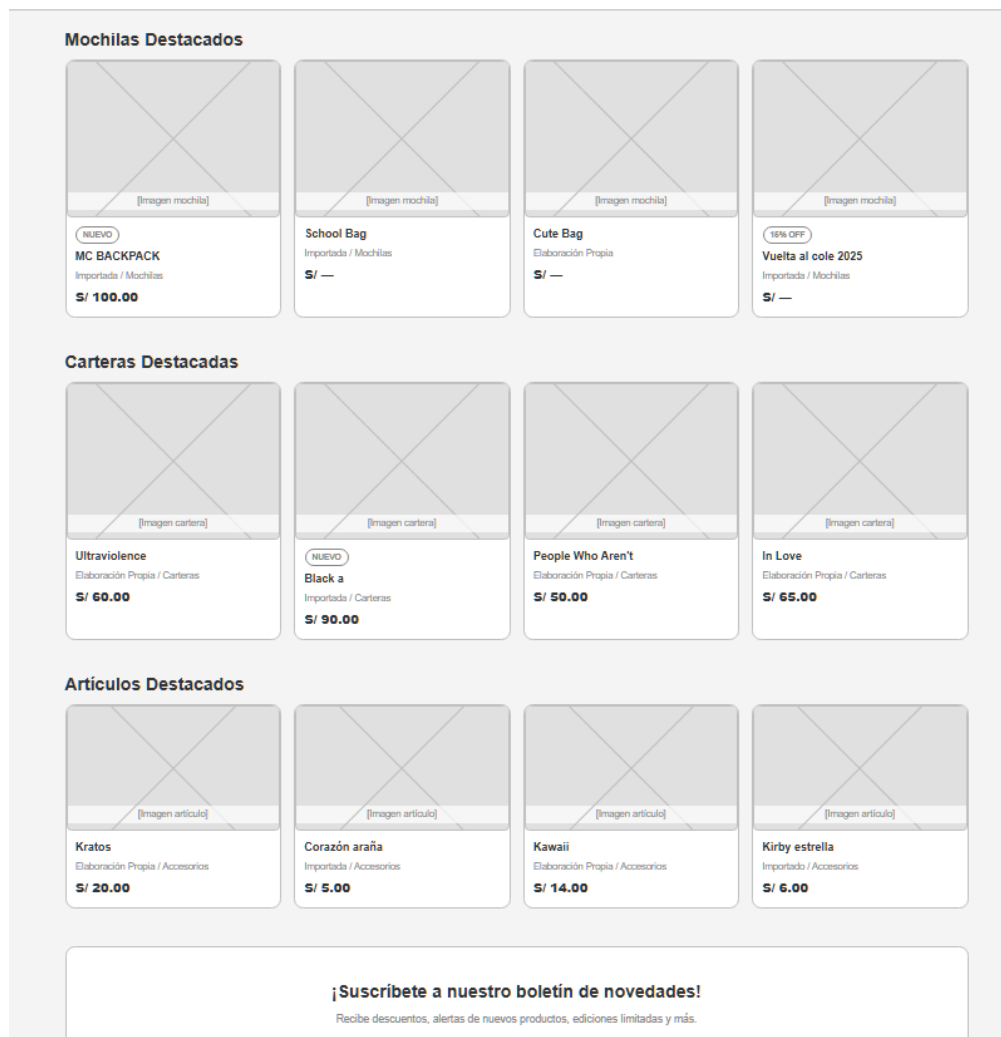


Figura 7
Wireframe segunda parte de la página de Inicio



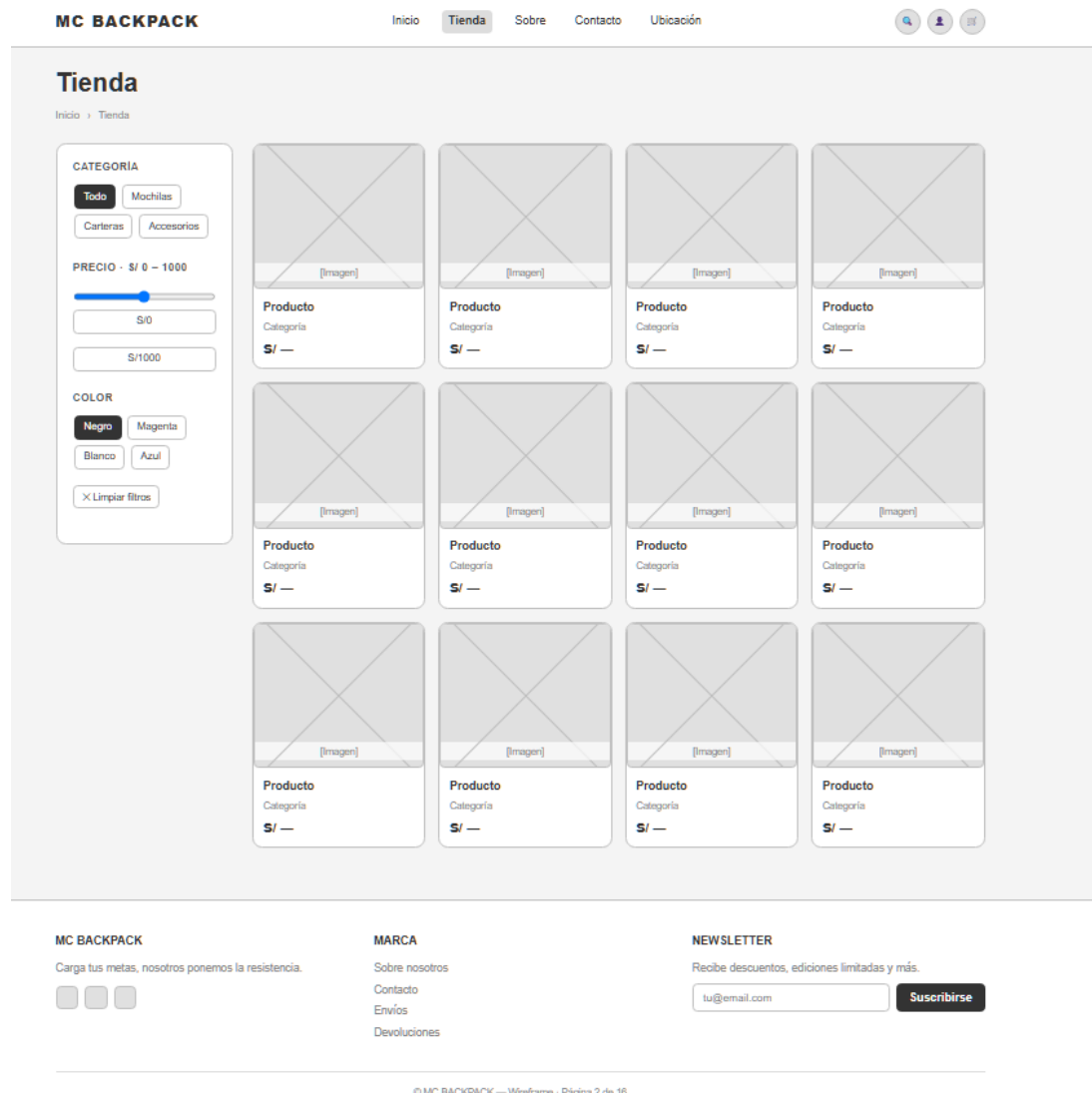
Nota. Diseño estructural de la página principal destacando el banner, las categorías y las cuadrículas de productos.

La interfaz de inicio establece la jerarquía visual del sitio, priorizando un banner atractivo seguido de la categorización de los artículos en confecciones propias e importadas. Se han distribuido secciones de productos destacados mediante cuadrículas organizadas para facilitar la exploración inmediata del catálogo, finalizando con una sección dedicada a la suscripción del boletín informativo.

Figura

8

Wireframe de la página de la Tienda

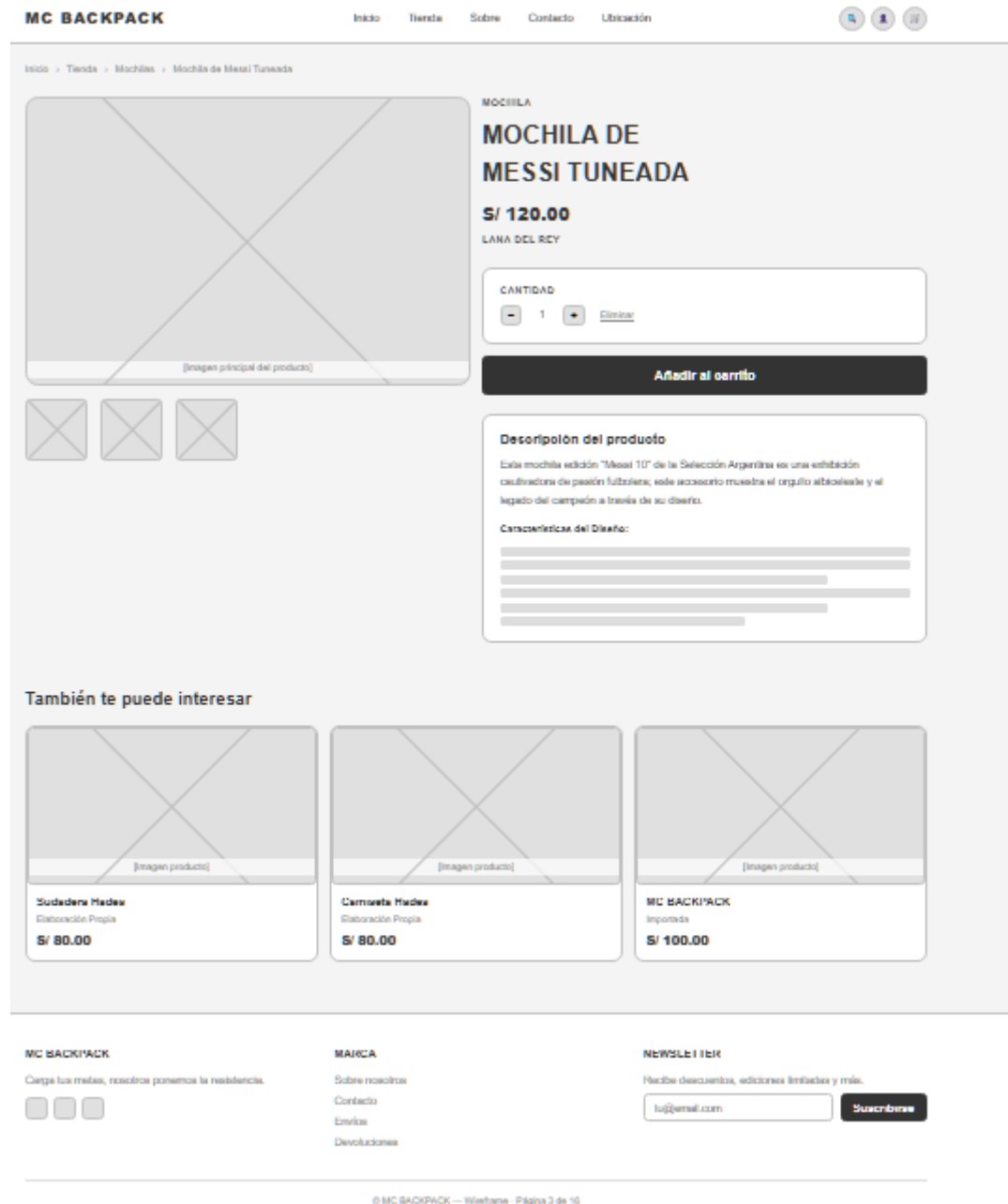


Nota. Estructura del catálogo con sistema de filtros laterales.

Esta vista organiza el inventario completo de la marca. Cuenta con una barra lateral izquierda funcional destinada a los filtros de búsqueda (categoría, precio y color), permitiendo al usuario refinar los resultados. La cuadrícula principal maximiza el área de visualización de los artículos, manteniendo la coherencia visual con la presentación de productos de la página inicial.

Figura 9

Wireframe de Información de Producto



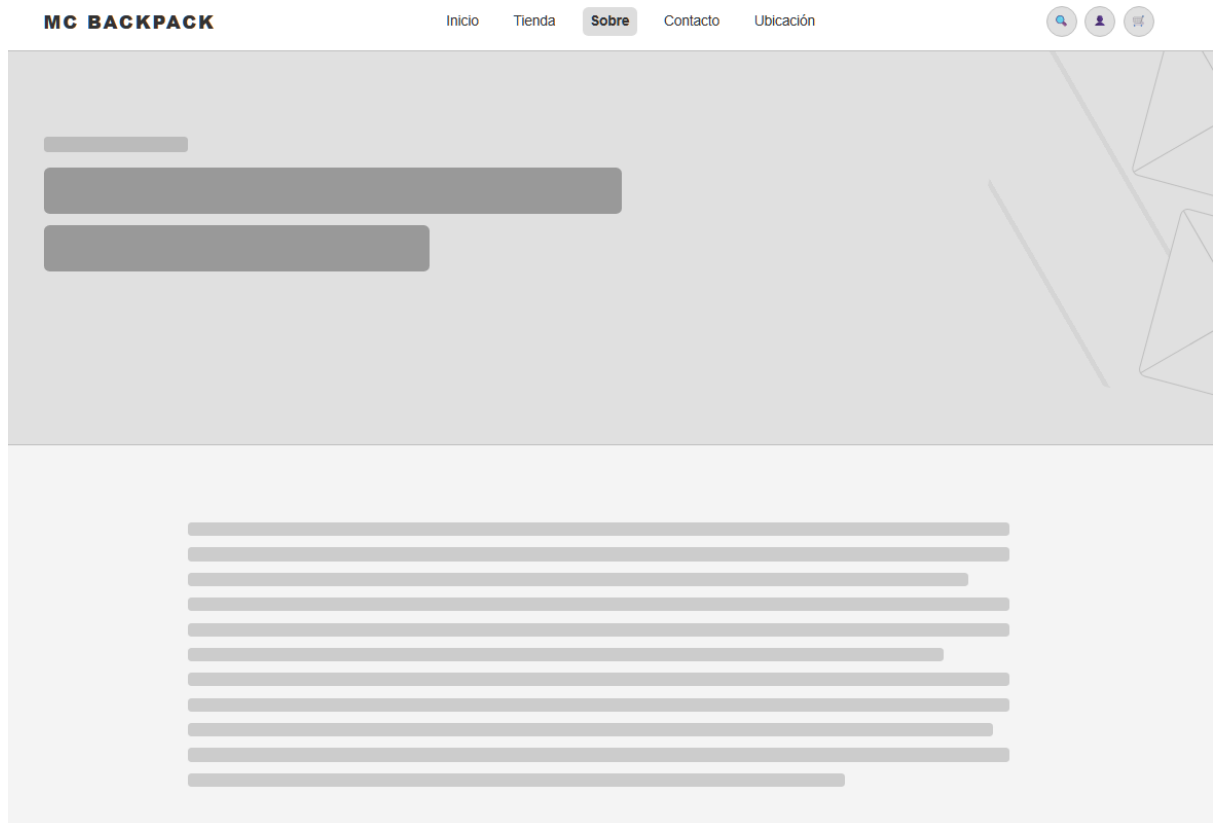
Nota. Vista detallada de un artículo individual con opciones de compra.

La estructura se divide en dos columnas principales para optimizar la toma de decisiones del comprador. La izquierda concentra la galería de imágenes del producto, mientras que la derecha agrupa la información crítica: precio, selector de cantidad, botón de llamada a

la acción principal para añadir al carrito y los detalles descriptivos de manufactura y diseño. Se incluye también una sección inferior de productos sugeridos para fomentar la venta cruzada.

Figura 10

Wireframe de la página Nosotros

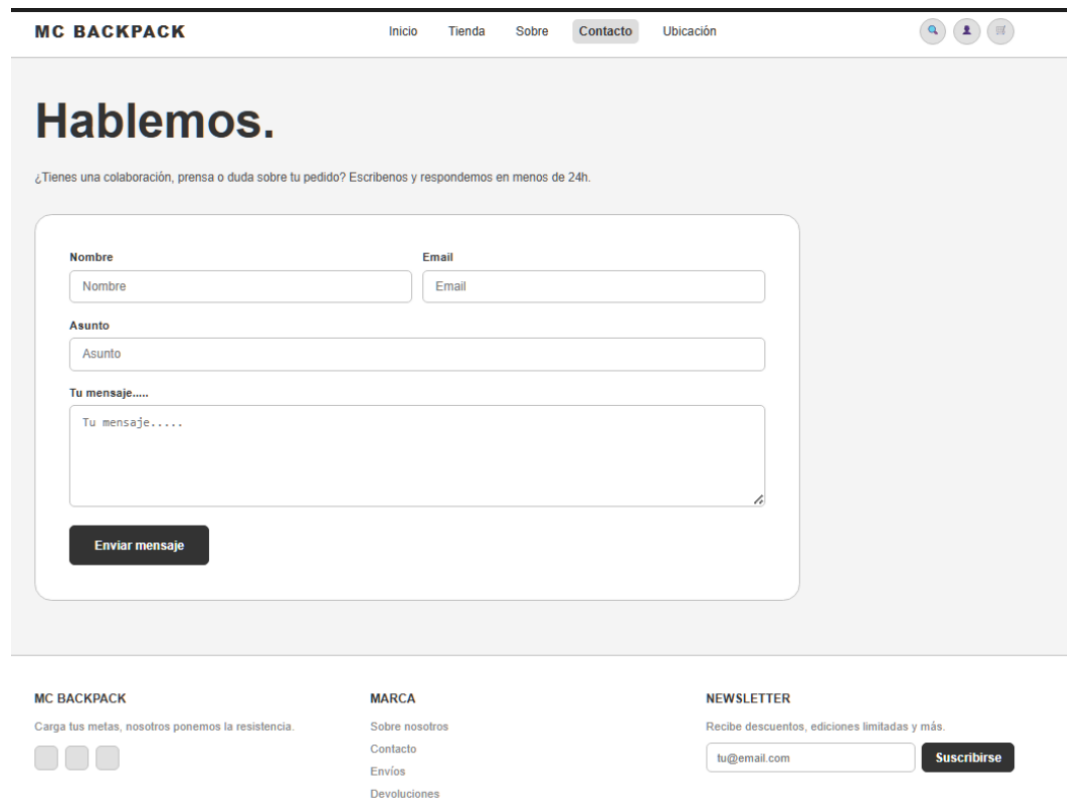


Nota. Maquetación de la historia y valores de la marca.

El diseño prioriza la narrativa visual e histórica de MC Backpack mediante un encabezado de ancho completo que enmarca el concepto de la marca. Debajo, un bloque de texto centralizado está dispuesto para albergar la historia sobre los orígenes del emprendimiento en Lima, el esfuerzo de su fundadora y la evolución del modelo de negocio, transmitiendo cercanía y resiliencia al visitante.

Figura 11

Wireframe de la página de Contacto



MC BACKPACK Inicio Tienda Sobre **Contacto** Ubicación

Hablemos.

¿Tienes una colaboración, prensa o duda sobre tu pedido? Escribenos y respondemos en menos de 24h.

Nombre Email

Asunto

Tu mensaje.....

Enviar mensaje

MC BACKPACK
Carga tus metas, nosotros ponemos la resistencia.


MARCA
Sobre nosotros
Contacto
Envíos
Devoluciones

NEWSLETTER
Recibe descuentos, ediciones limitadas y más.
 Suscribirse

Nota. Formulario de comunicación directa para atención al cliente.

Se emplea un enfoque minimalista y directo. Tras un breve encabezado introductorio, la pantalla se centra en un formulario estructurado e intuitivo que solicita únicamente los datos necesarios (nombre, correo, asunto y mensaje) para facilitar la resolución de dudas, devoluciones o consultas sobre colaboraciones.

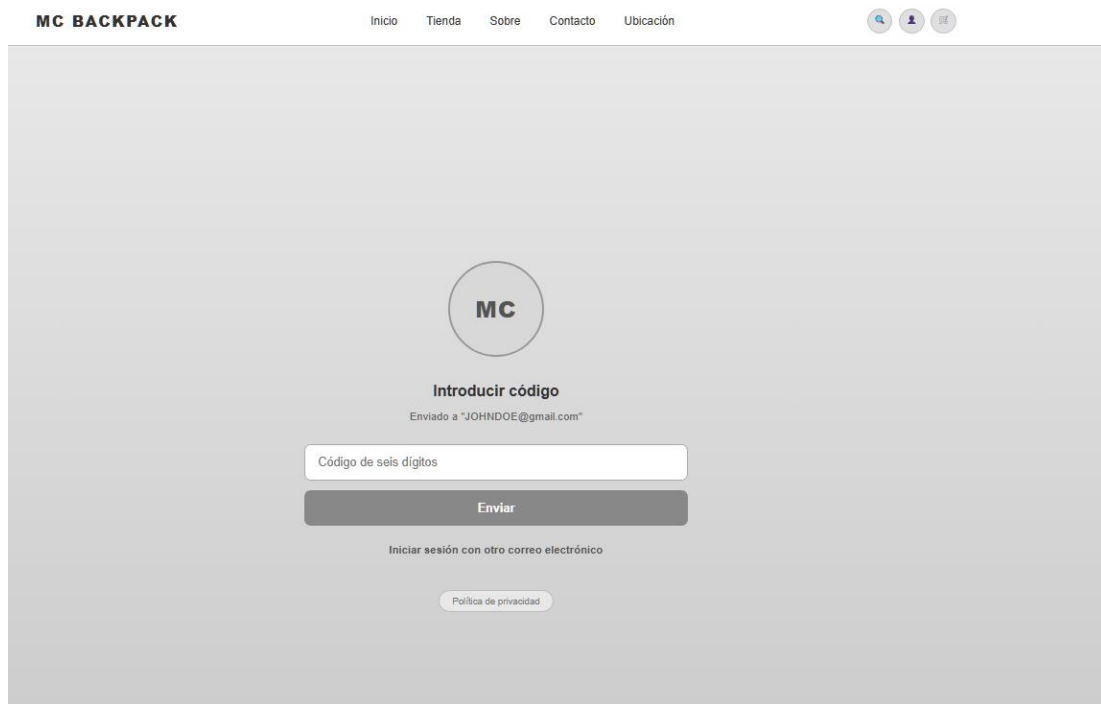
Figura 12

Wireframe de Registro (Paso 1: Correo)

Términos del servicio y la [Política de privacidad](#)'. At the bottom of the card is a link: '¿No tienes cuenta? [Crear cuenta](#) →'. The top of the page has a navigation bar with 'MC BACKPACK' on the left and links 'Inicio', 'Tienda', 'Sobre', 'Contacto', 'Ubicación' in the center. On the right of the navigation bar are three circular icons: a magnifying glass, a person, and a document." data-bbox="176 162 855 495"/>

Nota. Pantalla inicial del flujo de autenticación de usuarios.

El diseño del flujo de registro inicia con una interfaz limpia y libre de distracciones, presentando el logotipo centralizado y un único campo de entrada para el correo electrónico. Esta disposición busca reducir la fricción inicial del usuario al crear una cuenta o iniciar sesión.

Figura 13*Wireframe de Registro (Paso 2: Verificación)*

MC BACKPACK

Inicio Tienda Sobre Contacto Ubicación

MC

Introducir código

Enviado a "JOHNDOE@gmail.com"

Código de seis dígitos

Enviar

Iniciar sesión con otro correo electrónico

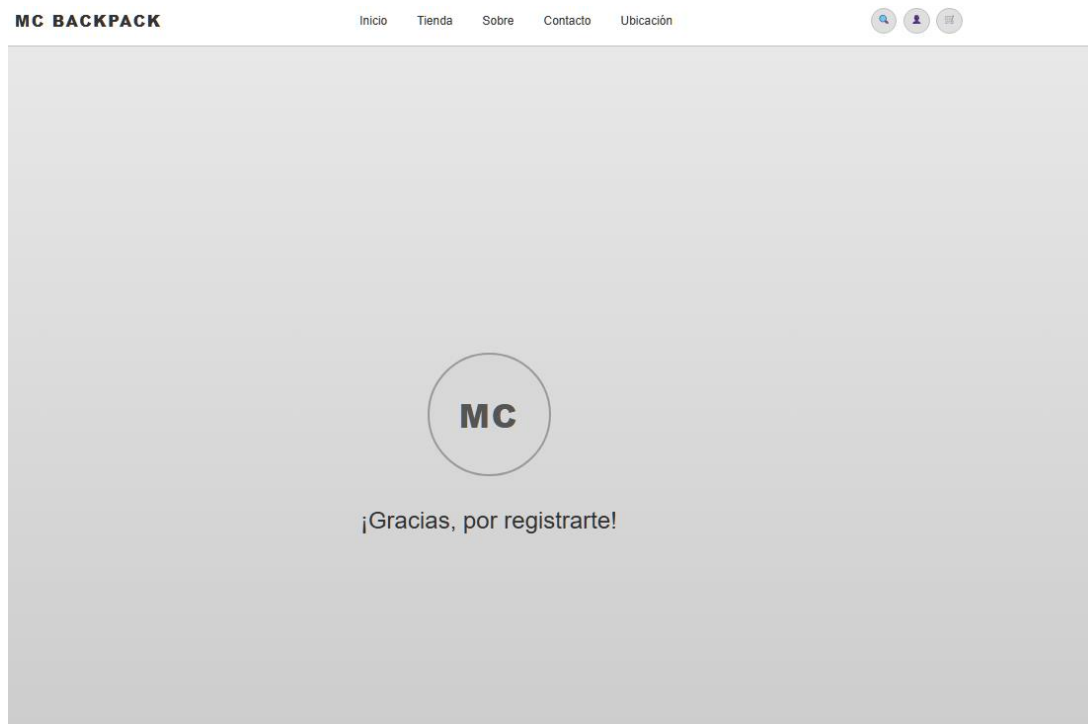
Política de privacidad

Nota. Pantalla de validación mediante código de seguridad.

Dando continuidad al proceso de autenticación, esta vista presenta un fondo oscurecido que resalta el componente central: el ingreso del código de verificación de seis dígitos. La estructura asegura que el usuario enfoque su atención en completar la validación de seguridad de su correo electrónico antes de proceder.

Figura 14

Wireframe de Registro (Paso 3: Éxito)



Nota. Pantalla de confirmación de creación de cuenta.

Se utiliza un diseño afirmativo y directo que confirma visualmente el éxito de la operación. La simplicidad del mensaje central y el logotipo refuerzan la finalización exitosa del registro, preparando al usuario para retornar a la experiencia de compra.

Figura 15*Wireframe de Términos de Servicio*

Nota. Estructura de visualización de información legal.

Orientado puramente a la legibilidad, el diseño elimina elementos gráficos innecesarios y presenta el texto en bloques jerarquizados con títulos claros. La estructura asegura que las políticas operativas, condiciones de compra y limitaciones de responsabilidad sean fácilmente leídas por el consumidor.

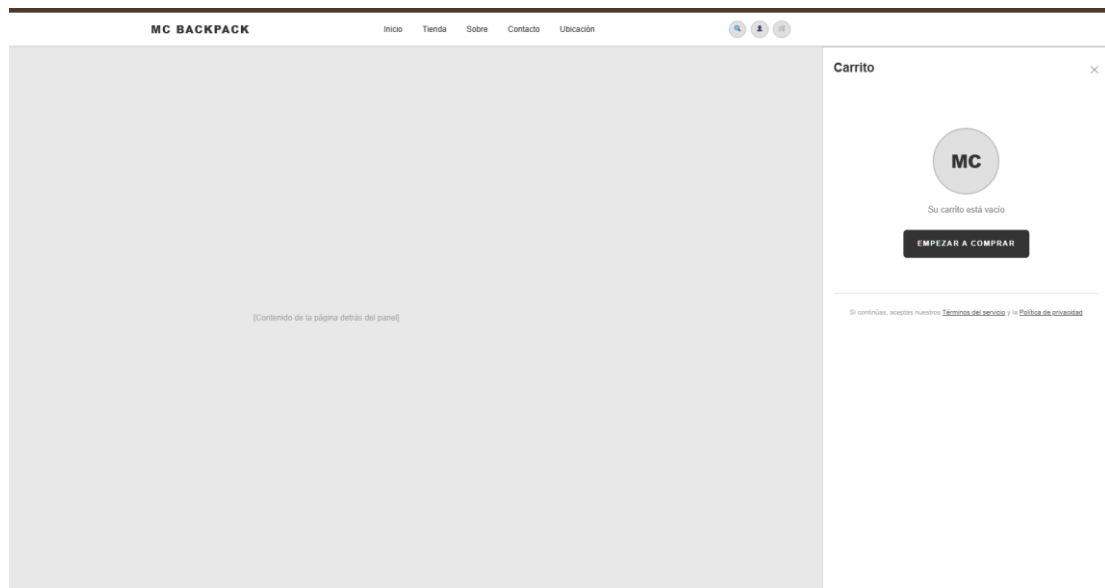
Figura 16

Wireframe de Políticas de Privacidad



Nota. Disposición del documento de protección de datos.

Manteniendo el formato de la página de términos, esta vista organiza los apartados sobre recopilación de información, uso de cookies y derechos del usuario en un formato de texto estructurado, garantizando la transparencia informativa del comercio electrónico.

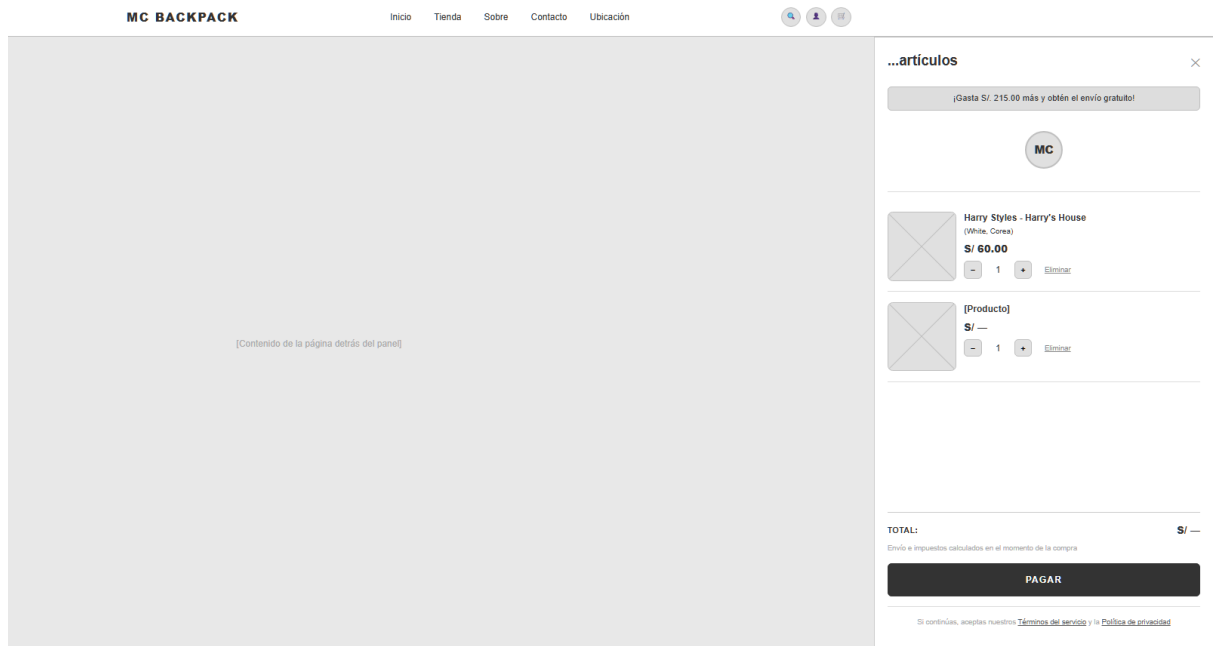
Figura 17*Wireframe del Carrito Vacío*

Nota. Estado inicial del panel lateral de compras.

El carrito se presenta como un panel lateral superpuesto. En su estado vacío, el diseño busca redirigir la atención del usuario nuevamente hacia el catálogo mediante un claro llamado a la acción para comenzar a comprar, optimizando así la retención y la conversión.

Figura 18

Wireframe de Carrito com Artículos



Nota. Panel lateral con listado de productos seleccionados y subtotal.

Cuando contiene productos, el panel lateral detalla de forma clara la imagen, precio y cantidad de cada ítem, permitiendo ajustes rápidos. Se incorpora un banner promocional estratégico en la parte superior para incentivar el alcance del envío gratuito, culminando con el resumen de costos y el botón hacia el pago.

Figura 19

Wireframe de Pago

Contacto [Iniciar sesión](#)

☐ Enviarme novedades y ofertas por correo electrónico

Entrega

Métodos de envío

Pago

Todas las transacciones son seguras y están encriptadas.

Mercado Pago

Se le redirigirá a Mercado Pago para que complete la compra.

Dirección de facturación

☐ La misma dirección de envío

☒ Usar una dirección de facturación distinta

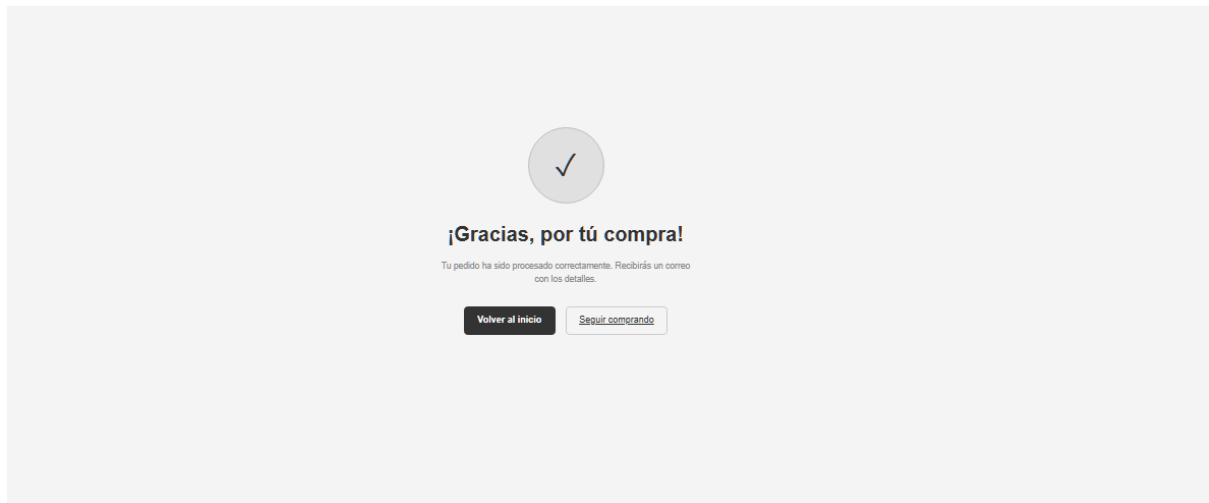
☐ Guardar mi información y consultar más rápidamente la próxima vez

☐ Enviarme novedades y ofertas por SMS

Pagar ahora

Nota. Formulario de finalización de compra y datos de envío.

La pantalla de checkout utiliza una distribución en dos columnas (formulario y resumen del pedido). El lado izquierdo desglosa metódicamente la recopilación de datos del cliente en secciones lógicas: contacto, dirección de entrega, métodos de envío y selección de pasarela de pago, manteniendo visible el resumen de la compra en la columna derecha para brindar transparencia continua sobre los costos.

Figura 20*Wireframe de Confirmación de Pago*

Nota. Pantalla de éxito tras finalizar una compra.

Esta vista transmite seguridad y agradecimiento tras la transacción. Su diseño centralizado con un ícono de verificación sirve para calmar cualquier ansiedad post-compra, ofreciendo además rutas claras de navegación para regresar al inicio o continuar explorando la tienda.

Figura 21

Wireframe de Ubicación



Nota. Diseño de la página informativa sobre tiendas físicas y puntos de entrega.

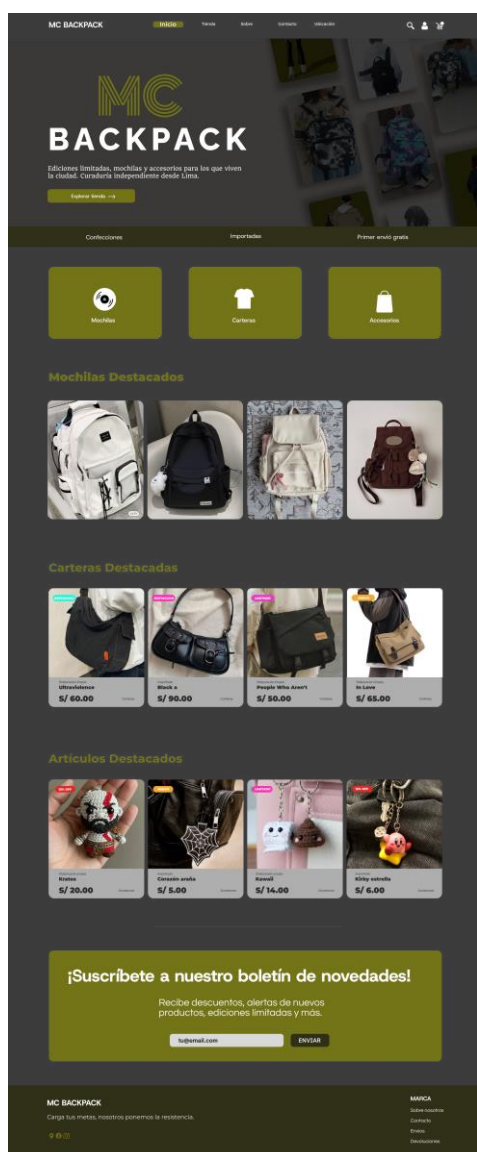
La vista se divide en secciones de alto contraste que detallan la presencia física de la marca. Incluye módulos para mostrar fotografías de la fachada, detalles exactos de dirección y horarios, así como un espacio dedicado a próximos eventos o ferias itinerantes. Se complementa con una zona reservada para la integración de un mapa interactivo que facilita la localización espacial por parte de los clientes.

ANEXO B: PROTOTIPOS FIGMA

Representación visual final de las interfaces de usuario aplicadas al emprendimiento.

Figura 22

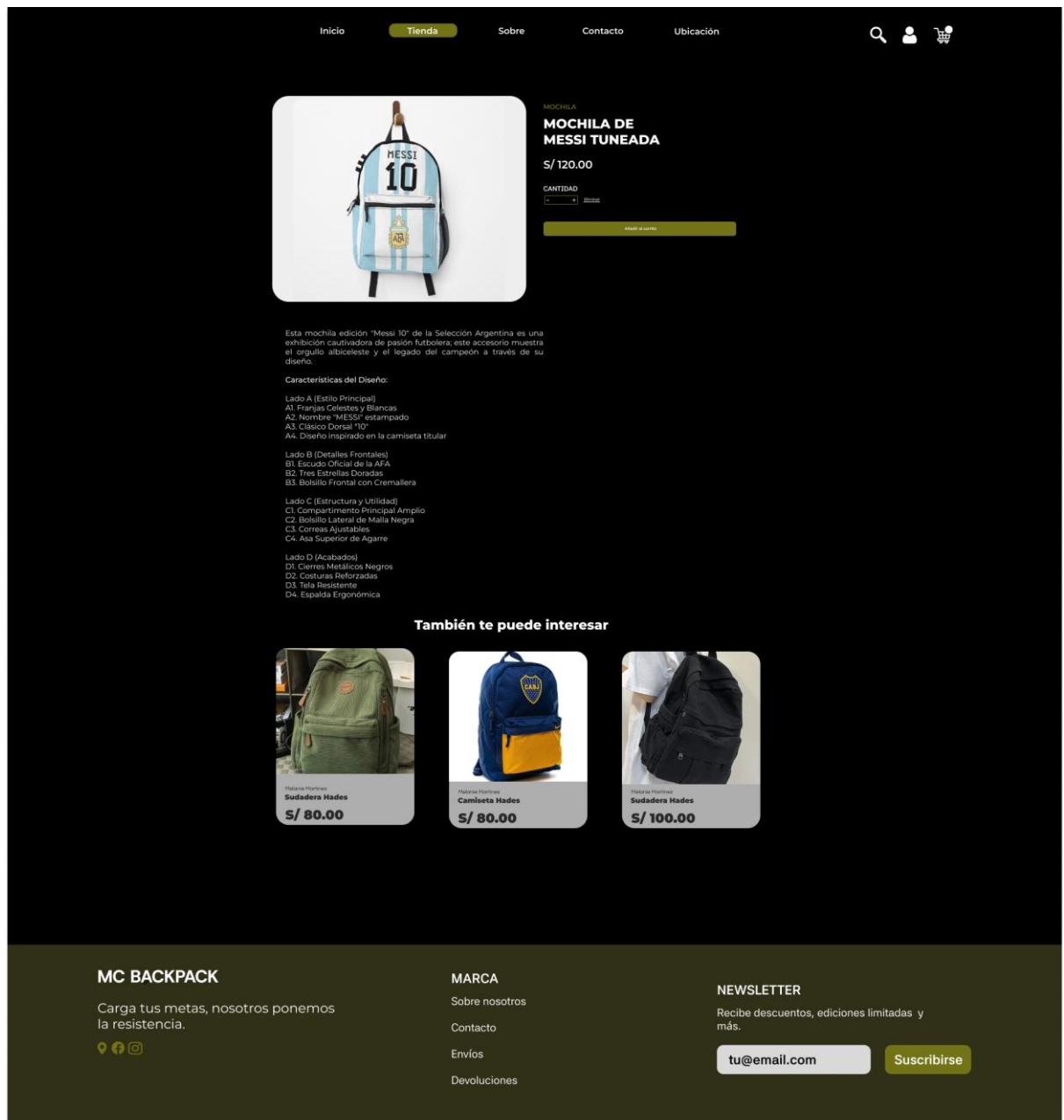
Prototipo de la Página de Inicio del Sistema



Nota. Diseño visual de la pantalla principal a la cual acceden los usuarios al ingresar a la plataforma, mostrando las opciones iniciales de navegación y promociones destacadas.

Figura 23

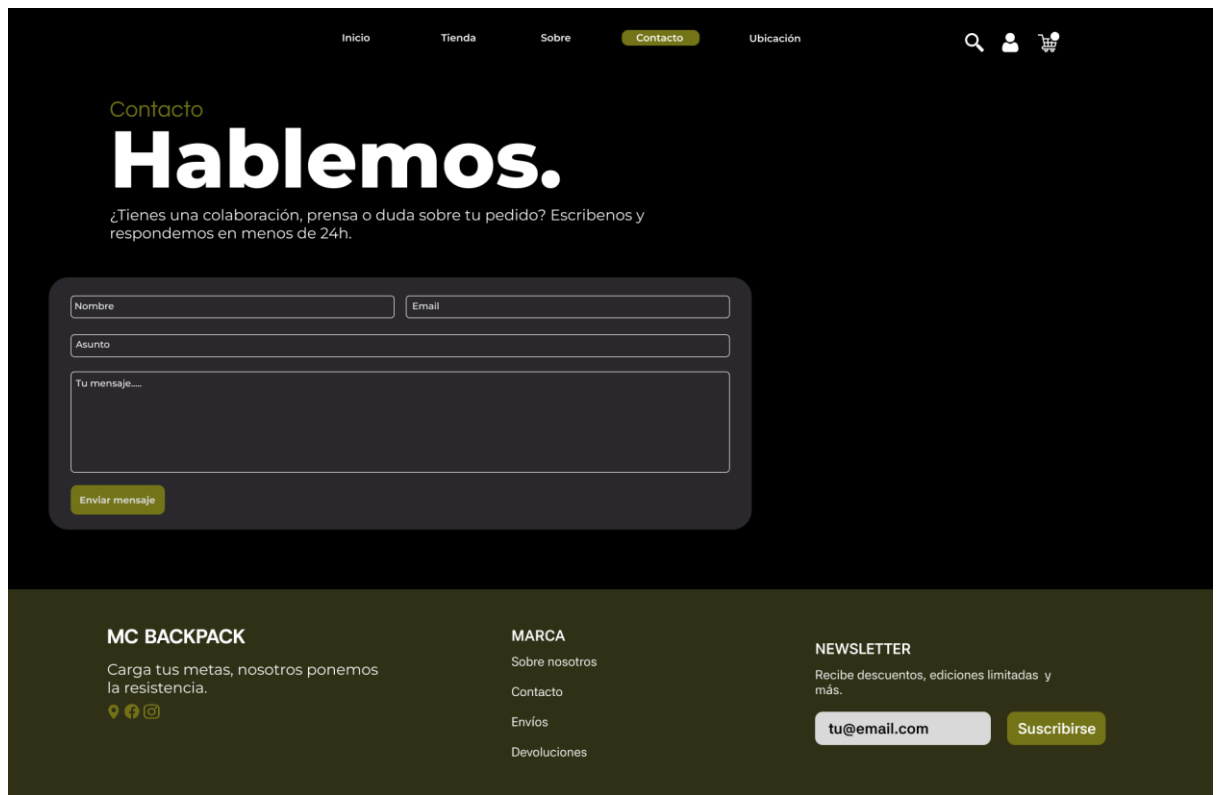
Prototipo de Información Detallada de Producto: Mochilas



Nota. Diseño visual de la interfaz de usuario que expone las características, especificaciones, precios y detalles adicionales específicos para el catálogo de mochilas.

Figura 24

Prototipo de Página Contacto y Atención al Cliente



Prototipo de la página de contacto y atención al cliente. La interfaz tiene un fondo oscuro con elementos en verde.

Encabezado: Menú de navegación (Inicio, Tienda, Sobre, **Contacto**, Ubicación), iconos de búsqueda, perfil y carrito.

Contenido Principal:

- Contacto** (etiqueta verde)
- Hablemos.** (título principal)
- Texto: ¿Tienes una colaboración, prensa o duda sobre tu pedido? Escríbenos y respondemos en menos de 24h.
- Formulario de contacto:**
 - Campos: Nombre, Email, Asunto.
 - Área de texto: Tu mensaje....
 - Botón: Enviar mensaje

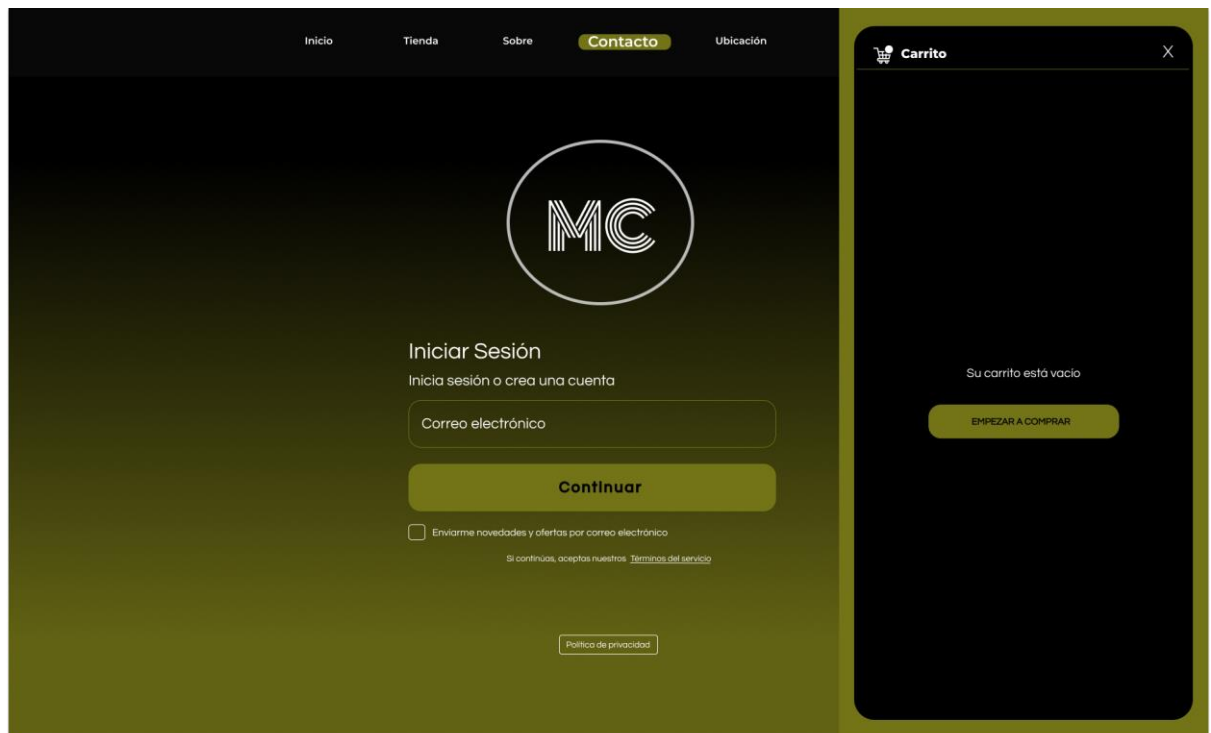
Pie de página:

- MC BACKPACK:** Carga tus metas, nosotros ponemos la resistencia. (iconos de ubicación, Facebook, Instagram)
- MARCA:** Sobre nosotros, Contacto, Envíos, Devoluciones
- NEWSLETTER:** Recibe descuentos, ediciones limitadas y más. Campo: Botón: **Suscribirse**

Nota. Diseño visual de la sección destinada a que los usuarios puedan comunicarse con el equipo de soporte técnico o atención al cliente de la plataforma.

Figura 25

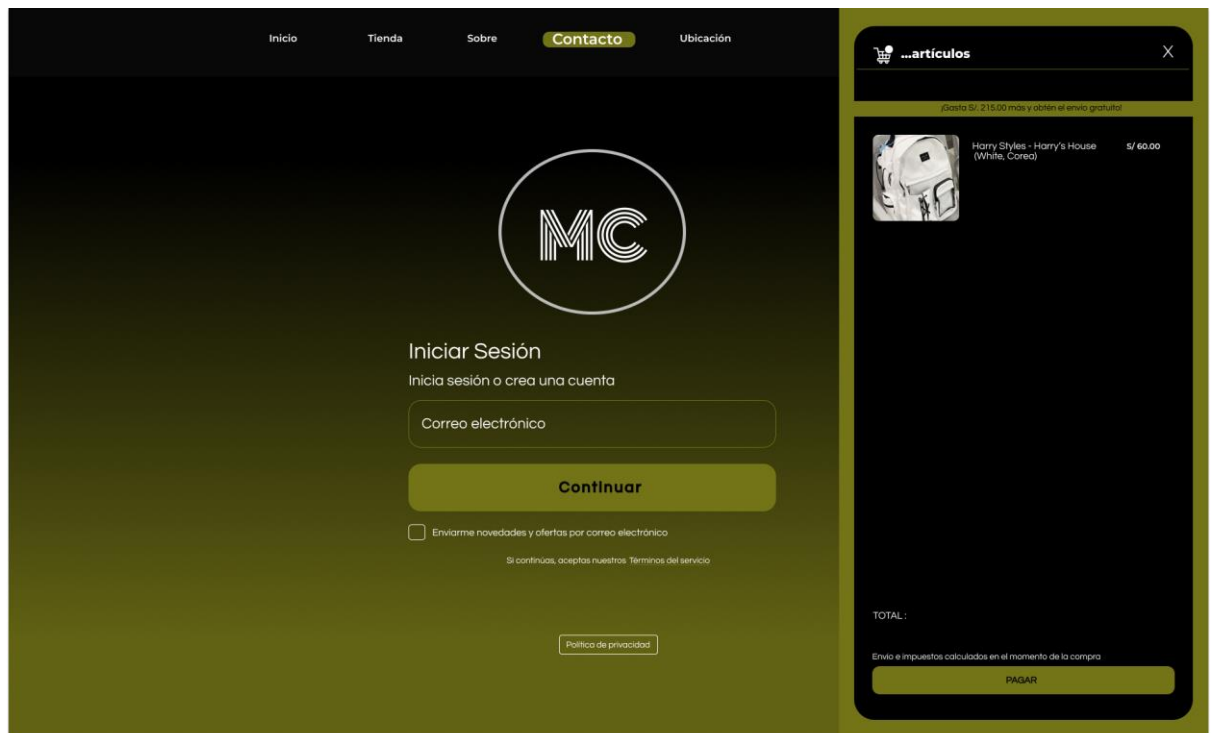
Prototipo de Interfaz de Carrito de Compras Vacío



Nota. Diseño visual de la interfaz de usuario que indica que el carrito de compras actualmente no contiene ningún producto seleccionado por el cliente.

Figura 26

Prototipo de Interfaz de Carrito de Compras Pedido Activo



Nota. Diseño visual de la interfaz de usuario donde se visualizan los productos que el cliente ha agregado al carrito, mostrando los detalles de la compra listos para el pago.

Figura 27

Prototipo de Interfaz de la Selección Informativa Sobre Nosotros



Nota. Diseño visual de la página corporativa que presenta la historia, misión, visión y los detalles generales del equipo de trabajo y la empresa al público.

Figura 28

Prototipo de Interfaz para el Ingreso de Datos de Facturación y Pago

MC

Contacto [Iniciar sesión](#)

Correo Electrónico

☐ Envíame novedades y ofertas por correo electrónico

Entrega

País/Región

Nombre Apellido

DNI

Dirección

Casa, apartamento, etc. (opcional)

Código postal (opcional) Ciudad

Región

Métodos de envío

Ingresar la dirección de envío para ver los métodos disponibles.

Pago

Todas las transacciones son seguras y están encriptadas.

Mercado Pago

Se te redirigirá a Mercado Pago para que completes la compra.

Dirección de facturación

☒ La misma dirección de envío

☐ Usar una dirección de facturación distinta

País/Región

Nombre Apellido

DNI

Dirección

Casa, apartamento, etc. (opcional)

Ciudad Región Código postal

Teléfono

☐ Guardar mi información y consultar más rápidamente la próxima vez

☐ Envíame novedades y ofertas por SMS

Pagar ahora

Nota. Diseño visual del formulario diseñado para que el usuario proporcione su información financiera, método de pago y detalles de facturación necesarios para completar la compra.

Figura 29

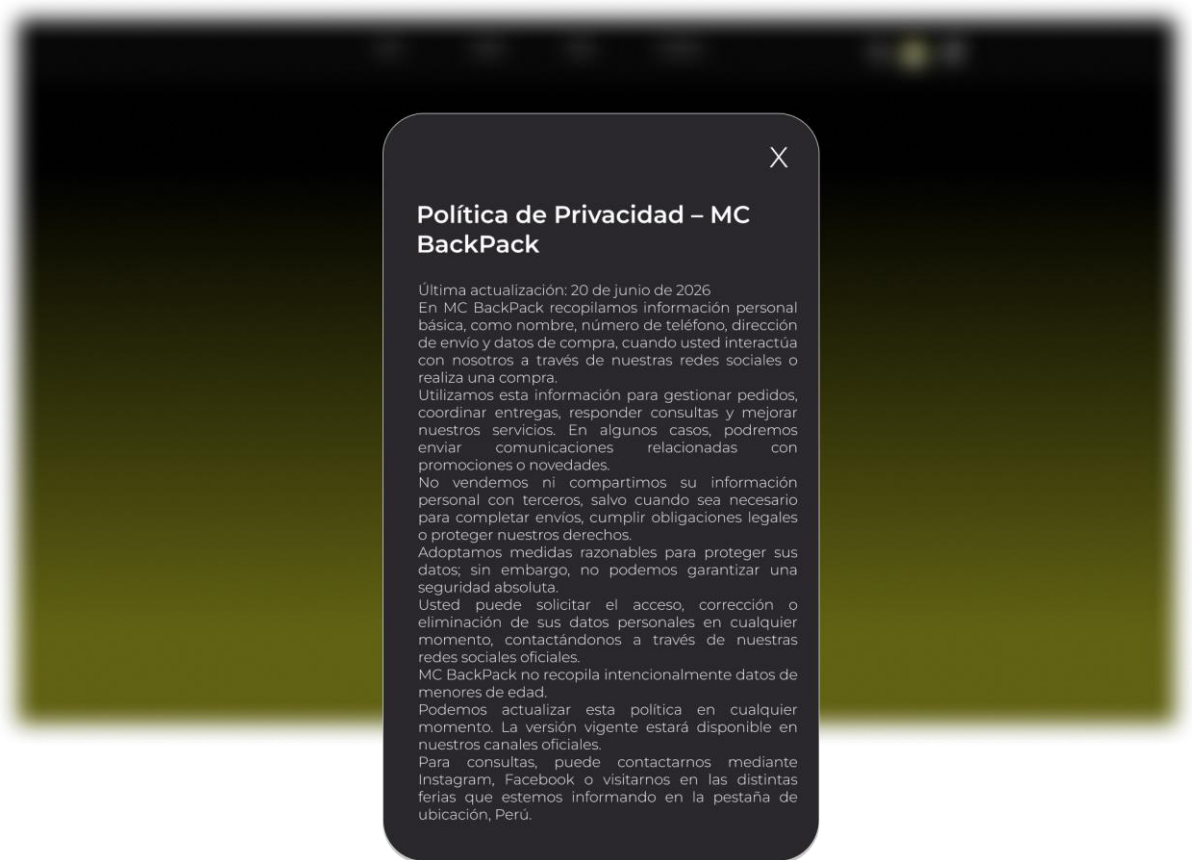
Prototipo de Interfaz de Confirmación de Pago y Transacción Exitosa



Nota. Diseño visual de la pantalla que notifica al usuario de forma clara que su transacción económica ha sido procesada y aprobada correctamente por la pasarela de pagos.

Figura 30

Prototipo de Interfaz del Documento de Políticas de Privacidad



Nota. Diseño visual de la sección legal y regulatoria que explica a los usuarios cómo se recopila, utiliza y protege su información personal dentro de la plataforma.

Figura 31

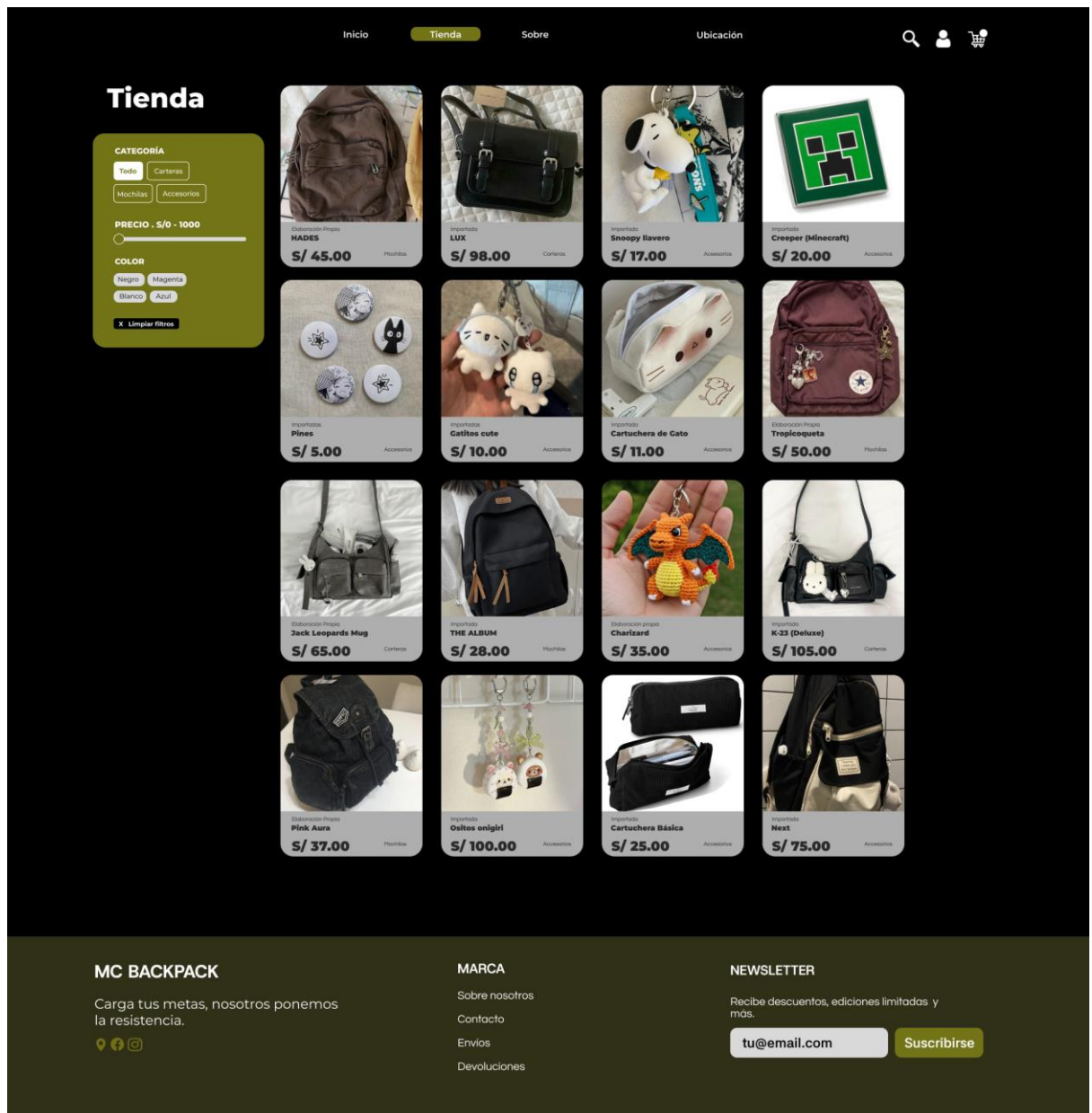
Prototipo de Interfaz del Documento de Términos y Condiciones de Servicio



Nota. Diseño visual de la página que estipula formalmente las reglas, lineamientos y acuerdos legales que rigen el uso del comercio electrónico por parte de los clientes.

Figura 32

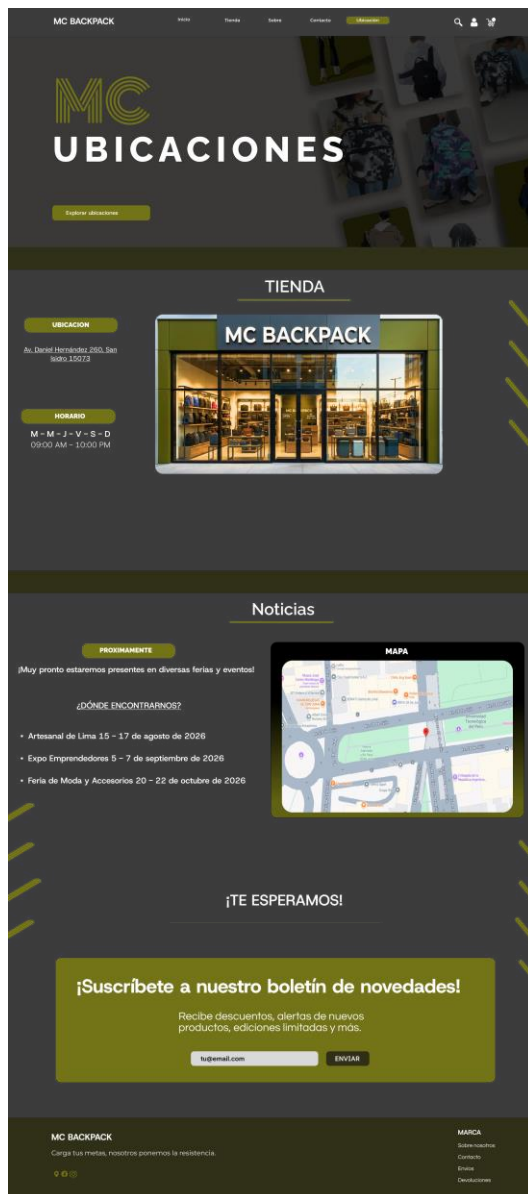
Prototipo de Interfaz del Catálogo Principal de la Tienda



Nota. Diseño visual del apartado central donde se exponen todos los productos disponibles para la venta, incluyendo sus respectivas herramientas de filtrado y categorías.

Figura 33

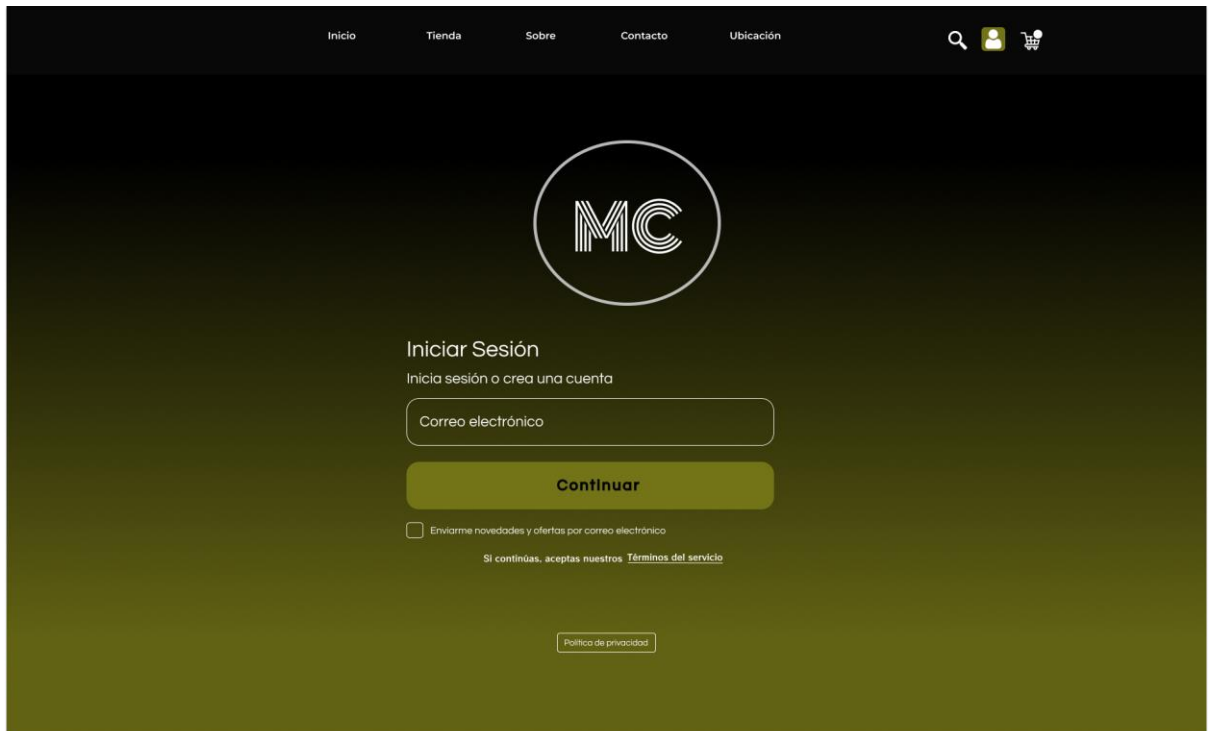
Prototipo de Interfaz de la Ubicación Geográfica de la Empresa



Nota. Diseño visual de la pantalla que integra un mapa interactivo de localización y detalla las direcciones físicas correspondientes a las sucursales o puntos de venta.

Figura 34

Prototipo de Interfaz del Formulario de Registro de Nuevos Usuarios



The image shows a web interface prototype for a new user registration form. The background is a dark gradient. At the top, there is a navigation bar with links: Inicio, Tienda, Sobre, Contacto, and Ubicación. On the right side of the navigation bar are icons for search, user profile, and shopping cart. In the center of the page is a circular logo with the letters 'MC'. Below the logo, the text 'Iniciar Sesión' is displayed, followed by 'Inicia sesión o crea una cuenta'. There is a text input field labeled 'Correo electrónico'. Below the input field is a green button labeled 'Continuar'. Under the button is a checkbox with the text 'Enviarme novedades y ofertas por correo electrónico'. At the bottom, there is a link 'Si continúas, aceptas nuestros [Términos del servicio](#)'. At the very bottom, there is a link 'Política de privacidad'.

Nota. Diseño visual de la vista inicial mediante la cual los nuevos visitantes pueden comenzar la creación de una cuenta en el sistema proporcionando sus credenciales de acceso.

Figura 35

Prototipo de Interfaz para la Validación de Código de Seguridad de Registro

El prototipo de interfaz para la validación de código de seguridad de registro presenta un diseño limpio y moderno. En la parte superior, hay una barra de navegación con los enlaces: Inicio, Tienda, Sobre, Contacto y Ubicación. A la derecha de esta barra se encuentran íconos para búsqueda, perfil de usuario y carrito de compras. El fondo principal es de color verde oscuro. En el centro, se muestra un logo circular con las letras 'MC'. Debajo del logo, el texto indica 'Introducir código' y 'Enviado a "JOHNDOE@gmail.com"'. Hay un campo de entrada para el 'Código de seis dígitos' y un botón verde con el texto 'Enviar'. Debajo del botón, se ofrece la opción de 'Iniciar sesión con otro correo electrónico'. En la parte inferior, hay un enlace para la 'Política de privacidad'.

Nota. Diseño visual de la pantalla de seguridad que solicita el ingreso del código de verificación enviado al correo del usuario para confirmar la autenticidad de su cuenta.

Figura 36

Prototipo de Interfaz de Confirmación de Cuenta de Usuario Creada Exitosamente



Nota. Diseño visual del mensaje de sistema que certifica que el proceso de registro ha culminado de forma satisfactoria, permitiendo al usuario iniciar sesión en su perfil.

ANEXO C1: IMPLEMENTACION DE HTML, CSS, BOOTSTRAP

Figura 37

Bootstrap, Google Fonts y CSS propios separados

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <meta name="description" content="MC Backpack - Mochilas importadas y creadas a tu estilo. Ediciones limitadas, mochilas y accesorios para los que
7      <title>MC BACKPACK - Mochilas y Accesorios</title>
8
9      <!-- Google Fonts (exactas del Figma) -->
10     <link rel="preconnect" href="https://fonts.googleapis.com">
11     <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
12     <link href="https://fonts.googleapis.com/css2?family=Host+Grotesk:wght@400;500;600;700;800&family=Monoton&family=Montserrat:wght@300;400;500;600;800;
13     900&family=Questrial&family=Raleway:wght@600;800;900&family=Quando&display=swap" rel="stylesheet">
14
15     <!-- Bootstrap CSS -->
16     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
17
18     <!-- Custom CSS -->
19     <link rel="stylesheet" href="css/style.css">
20     <link rel="stylesheet" href="css/cart.css">
21 </head>

```

Nota. Muestra Bootstrap 5.3.2 CDN (L15), Google Fonts (L9–12) y los 2 CSS propios separados (style.css + cart.css).

Figura 38

Aria, hamburger e iconos del navbar

```

20 </head>
21 <body>
22   <a href="#main-content" class="visually-hidden-focusable">Saltar al contenido principal</a>
23
24   <!-- ===== NAVBAR ===== -->
25   <nav class="navbar-custom" aria-label="Navegación Principal">
26     <div class="navbar-inner">
27       <!-- Logo MC -->
28       <div class="navbar-logo">
29         <div class="logo-circle" aria-label="Logo MC">
30           <span class="logo-text">MC</span>
31         </div>
32       </div>
33
34       <!-- Hamburger for mobile -->
35       <button class="hamburger" id="hamburger" aria-label="Abrir menú" aria-expanded="false">
36         <span></span><span></span><span></span></span>
37       </button>
38
39       <!-- Nav Links -->
40       <ul class="navbar-links" id="navbarLinks">
41         <li><a href="index.html" class="nav-link-custom active" aria-current="page">Inicio</a></li>
42         <li><a href="tienda.html" class="nav-link-custom">Tienda</a></li>
43         <li><a href="nosotros.html" class="nav-link-custom">Sobre</a></li>
44         <li><a href="contacto.html" class="nav-link-custom">Contacto</a></li>
45         <li><a href="ubicacion.html" class="nav-link-custom">Ubicación</a></li>
46       </ul>
47
48       <!-- Nav Icons -->
49       <div class="navbar-icons">
50         <button type="button" class="theme-toggle-btn" id="themeToggle" aria-label="Cambiar a modo claro">
51           <span class="theme-icon theme-icon-dark" aria-hidden="true">&#9728;</span>
52           <span class="theme-icon theme-icon-light" aria-hidden="true">&#9789;</span>
53         </button>
54         <div class="navbar-search-wrap">
55           <button type="button" class="nav-icon-btn" id="searchToggle" aria-label="Buscar" aria-expanded="false">
56             
57           </button>
58           <form class="navbar-search-form" id="navbarSearchForm" role="search">
59             <input type="search" id="navbarSearchInput" class="navbar-search-input" placeholder="Buscar productos..." aria-label="Buscar productos">
60             <button type="submit" class="navbar-search-submit" aria-label="Buscar">&#8594;</button>
61           </form>
62         </div>
63         <a href="registro.html" aria-label="Mi cuenta"></a>
64         <button type="button" class="nav-cart-btn" id="cartToggle" aria-label="Carrito de compras">
65           
66           <span class="cart-badge d-none" id="cartBadge">0</span>
67         </button>
68       </div>
69     </div>
70   </nav>

```

Nota. Demostración de `<nav aria-label>`, `<ul>`, `<button aria-expanded>`, `aria-current="page"`. HTML semántico y accesible en un solo bloque.

Figura 39

Main, Section y Article con data

```

72 <main id="main-content">
73   <!-- ===== HERO SECTION (Inicio) ===== -->
74   <section id="inicio" class="hero-section">
75     <div class="hero-content">
76       <h1 class="hero-mc">MC</h1>
77       <h2 class="hero-backpack">BACKPACK</h2>
78       <p class="hero-description" id="greeting-message">Ediciones limitadas, mochilas y accesorios para los que viven la ciudad. Curaduría independiente desde Lima.</p>
79       <div class="hero-cta">
80         <a href="#tienda" class="hero-link">Explorar tienda</a>
81         <span class="hero-arrow"></span>
82       </div>
83     </div>
84
85     <!-- Category pills -->
86     <div class="hero-categories">
87       <div class="category-pill">
88         <span>Confecciones</span>
89       </div>
90       <div class="category-pill">
91         <span>Importadas</span>
92       </div>
93       <div class="category-pill">
94         <span>Primer envío gratis</span>
95       </div>
96     </div>
97
98     <!-- MC BACKPACK badge + CTA -->
99     <div class="hero-badge">
100       <span class="badge-text">MC BACKPACK</span>
101       <a href="#tienda" class="btn-explore" role="button">Ver Catálogo</a>
102     </div>
103   </section>
104
105   <!-- ===== MOCHILAS DESTACADOS ===== -->
106   <section id="mochilas" class="section-products">
107     <div class="section-container">
108       <h2 class="section-title">Mochilas Destacados</h2>
109
110       <!-- Búsqueda -->
111       <div class="search-bar-container">
112         <input type="text" id="searchInput" class="search-input" placeholder="Buscar productos..." aria-label="Buscar productos">
113       </div>
114
115       <div class="products-grid" id="product-list">
116         <!-- Mochila 1 -->
117         <article class="product-card product-item" data-category="mochilas" data-name="School Bags" data-id="mochila-1">
118           <div class="product-img-wrapper">
119             
120           </div>
121           <div class="product-info">
122             <span class="product-origin">Elaboración Propia</span>
123             <h3 class="product-name">Sudadera Hades</h3>
124             <div class="product-bottom">
125               <span class="product-price">$ / 80.00</span>
126               <span class="product-cat">Mochilas</span>
127             </div>
128             <button type="button" class="add-to-cart-btn">Añadir al carrito</button>
129           </div>
130         </article>

```

Nota. <main id="main-content"> (L72), <section id="inicio"> (L74), primer <article class="product-card"> con data-category y data-id (L117). Semántica HTML5 clara.

Figura 40

Footer y scripts al final del body

```

359 <!-- ===== FOOTER ===== -->
360 <footer class="footer-section">
361   <div class="footer-container">
362     <div class="footer-left">
363       <h3 class="footer-brand">MC BACKPACK</h3>
364       <p class="footer-slogan">Carga tus metas, nosotros ponemos la resistencia.</p>
365       <div class="footer-socials">
366         <a href="ubicacion.html" aria-label="Ubicación"></a>
367         <a href="#" aria-label="Facebook"></a>
368         <a href="#" aria-label="Instagram"></a>
369       </div>
370     </div>
371     <div class="footer-right">
372       <h4 class="footer-heading">MARCA</h4>
373       <ul class="footer-links">
374         <li><a href="nosotros.html">Sobre nosotros</a></li>
375         <li><a href="contacto.html">Contacto</a></li>
376         <li><a href="#">Envíos</a></li>
377         <li><a href="#">Devoluciones</a></li>
378         <li><a href="terminos.html">Términos</a></li>
379         <li><a href="politicas.html">Políticas</a></li>
380       </ul>
381     </div>
382   </div>
383 </footer>
384
385 <!-- Botón Volver Arriba -->
386 <button id="backToTop" class="back-to-top d-none" aria-label="Volver arriba"></button>
387
388 <!-- Bootstrap Bundle JS -->
389 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
390
391 <!-- Custom JS -->
392 <script src="js/theme.js"></script>
393 <script src="js/search.js"></script>
394 <script src="js/cart.js"></script>
395 <script src="js/main.js"></script>
396 </body>
397 </html>

```

Nota. Cierre semántico con <footer>, de links internos, y el bloque de <script> en

orden: theme.js → search.js → cart.js → main.js. Muestra carga modular de JS.

Figura 41

3 Hojas CSS separadas importadas en orden

```
14 <!-- Bootstrap CSS -->
15 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
16
17 <!-- Custom CSS -->
18 <link rel="stylesheet" href="css/style.css">
19 <link rel="stylesheet" href="css/cart.css">
20 <link rel="stylesheet" href="css/tienda.css">
```

Nota. Bootstrap.min.css, style.css, cart.css, tienda.css — evidencia directa de CSS modular con hoja propia por página.

Figura 42

Main, section y aria label

```

68 <main id="main-content">
69   <!-- ===== TIENDA HEADER ===== -->
70   <section class="tienda-header">
71     <div class="tienda-header-content">
72       <h1 class="tienda-title">Tienda</h1>
73     </div>
74   </section>
75
76   <!-- ===== TIENDA BODY ===== -->
77   <section class="tienda-body">
78     <div class="tienda-layout">
79       <!-- Sidebar Filtros -->
80       <aside class="tienda-sidebar" aria-label="Filtros de productos">
81         <h2 class="sidebar-label">BUSCAR</h2>
82         <input type="search" id="shopSearchInput" class="sidebar-search-input" placeholder="Buscar por nombre..." aria-label="Buscar productos por nombre">
83
84         <h2 class="sidebar-label sidebar-label-mt">CATEGORÍA</h2>
85
86         <div class="sidebar-tags">
87           <button class="tag-btn tag-active" data-filter="all">Todos</button>
88           <button class="tag-btn" data-filter="mochilas">Mochilas</button>
89           <button class="tag-btn" data-filter="carteras">Carteras</button>
90           <button class="tag-btn" data-filter="accesorios">Accesorios</button>
91         </div>
92
93         <!-- Price range slider -->
94         <div class="sidebar-price">
95           <label for="priceRange" class="price-label">Precio máximo: <span id="priceValue">$/ 200</span></label>
96           <input type="range" class="price-slider" id="priceRange" min="0" max="200" value="200" aria-label="Filtro de precio máximo">
97           <div class="price-track">
98             <div class="price-fill" id="priceFill"></div>
99           </div>
100         </div>
101       </aside>

```

Nota. <aside aria-label="Filtros de productos"> (L80) + <label for="priceRange"> (L95) + slider con aria-label (L96). Semántica + accesibilidad + filtro interactivo visibles juntos.

Figura 43

Variables CSS – Paleta Completa del Figma

```

1  ✓ /* =====
2      MC BACKPACK - Custom Styles (Figma-Accurate)
3      ===== */
4
5  /* ===== CSS VARIABLES (Figma Palette) ===== */
6  ✓ :root {
7      /* Colors */
8      --black: #000000;
9      --dark: #060606;
10     --dark-bg: #0d0d0d;
11     --grey-dark: #292929;
12     --grey: #414141;
13     --grey-light: #AEAEAE;
14     --white: #FFFFFF;
15     --off-white: #FCFCFC;
16
17     --primary: #303017;
18     --primary-light: #737417;
19     --primary-accent: rgba(97, 98, 19, 1);
20
21     --accent-red: #DE2A2A;
22     --accent-red-bright: #9E2424;
23     --accent-pink: #FA2EB6;
24     --accent-yellow: #F4AA31;
25
26     --card-bg: #AEAEAE;
27     --card-img-bg: #414141;
28
29     /* Fonts */
30     --font-heading: 'Monoton', cursive;
31     --font-primary: 'Montserrat', sans-serif;
32     --font-body: 'Questrial', sans-serif;
33     --font-accent: 'Host Grotesk', sans-serif;
34     --font-brand: 'Raleway', sans-serif;
35     --font-quando: 'Quando', serif;
36
37     /* Spacing */
38     --page-padding: 165px;

```

Nota. Todo el sistema de tokens: --primary #303017, --primary-light #737417, tipografías, spacing --page-padding. Demuestra diseño basado en variables CSS, no valores hardcodedos.

Figura 44

Bloque RESPONSIVE – 3 media queries mobile-first

```

856  /* ===== RESPONSIVE ===== */
857  @media (max-width: 1200px) {
858      :root {
859          --page-padding: 80px;
860      }
861
862      .products-grid {
863          grid-template-columns: repeat(3, 1fr);
864      }
865  }
866
867  @media (max-width: 992px) {
868      :root {
869          --page-padding: 40px;
870      }
871
872      .products-grid {
873          grid-template-columns: repeat(2, 1fr);
874      }
875
876      .hero-categories {
877          gap: 40px;
878          flex-wrap: wrap;
879          justify-content: center;
880      }
881
882      .navbar-links {
883          display: none;
884          flex-direction: column;
885          position: fixed;
886          top: 90px;
887          left: 0;
888          right: 0;
889          background: rgba(0, 0, 0, 0.95);
890          padding: 24px;
891          gap: 16px;
892      }
893
894      .navbar-links.show {
895          display: flex;
896      }
897
898      .hamburger {
899          display: flex;
900      }
901  }

```

Nota. @media (max-width: 1200px) (L857), @media (max-width: 992px) (L867) con navbar mobile + hamburger, @media (max-width: 576px) con grid 1 columna. Demuestra enfoque responsive.

Figura 45

Bloque MODO CLARO – sistema data-theme

```

1053  /* =====
1054      MODO CLARO (híbrido): navbar, heroes y bloques de marca
1055      (footer, newsletter, badge) permanecen oscuros como en el
1056      Figma. Solo las secciones de contenido cambian a fondo claro.
1057      ===== */
1058  html[data-theme="light"] body {
1059      background-color: #f7f6f1;
1060  }
1061
1062  html[data-theme="light"] .section-products {
1063      background-color: #f7f6f1;
1064  }
1065
1066  html[data-theme="light"] .search-input {
1067      color: #1c1c12;
1068  }
1069
1070  html[data-theme="light"] .search-input::placeholder {
1071      color: #8a8a7a;
1072  }
1073
1074  html[data-theme="light"] .filter-btn {
1075      color: #1c1c12;
1076      border-color: #1c1c12;
1077  }
1078
1079  html[data-theme="light"] .filter-btn.active,
1080  html[data-theme="light"] .filter-btn:hover {
1081      background-color: #1c1c12;
1082      color: #f7f6f1;
1083  }

```

Nota. Todo el bloque `html[data-theme="light"]`: toggle de iconos (L1045–1050), `body` + sections (L1058–1083) vDemuestra theming avanzado con atributo HTML.

ANEXO C2: INTERACTIVIDAD CON JAVASCRIPT

Figura 46

Storage, getcart, safecart, addtocart

```

7  const CART_KEY = 'mc_cart';
8  const FREE_SHIPPING_THRESHOLD = 215;
9
10 function getCart() {
11   try {
12     return JSON.parse(localStorage.getItem(CART_KEY)) || [];
13   } catch (e) {
14     return [];
15   }
16 }
17
18 function saveCart(cart) {
19   localStorage.setItem(CART_KEY, JSON.stringify(cart));
20   renderCartDrawer();
21 }
22
23 function addToCart(item) {
24   const cart = getCart();
25   const existing = cart.find(i => i.id === item.id);
26   if (existing) {
27     existing.qty += 1;
28   } else {
29     cart.push({ ...item, qty: 1 });
30   }
31   saveCart(cart);
32   openCartDrawer();
33 }
34
35 function removeFromCart(id) {
36   const cart = getCart().filter(i => i.id !== id);
37   saveCart(cart);
38 }
39
40 function updateQty(id, delta) {
41   const cart = getCart();
42   const item = cart.find(i => i.id === id);
43   if (!item) return;
44   item.qty += delta;
45   if (item.qty <= 0) {
46     removeFromCart(id);
47     return;
48   }
49   saveCart(cart);

```

Nota. localStorage.getItem / setItem (L12, L19), lógica de qty (L25–32), cálculo matemático subtotal con reduce() Funcionalidades 1–5 del conteo.

Figura 47

InjectCartDrawer() – creación del DOM en tiempo de ejecución

```

58 function injectCartDrawer() {
59   if (document.getElementById('cartDrawer')) return;
60
61   const overlay = document.createElement('div');
62   overlay.className = 'cart-overlay';
63   overlay.id = 'cartOverlay';
64
65   const drawer = document.createElement('aside');
66   drawer.className = 'cart-drawer';
67   drawer.id = 'cartDrawer';
68   drawer.setAttribute('aria-label', 'Carrito de compras');
69   drawer.innerHTML = `
70     <div class="cart-drawer-header">
71       <span class="cart-drawer-title">Carrito</span>
72       <button type="button" class="cart-drawer-close" id="cartDrawerClose" aria-label="Cerrar carrito">&times;</button>
73     </div>
74     <div class="cart-drawer-body" id="cartDrawerBody"></div>
75     <div class="cart-drawer-footer d-none" id="cartDrawerFooter"></div>
76   `;
77
78   document.body.appendChild(overlay);
79   document.body.appendChild(drawer);
80
81   overlay.addEventListener('click', closeCartDrawer);
82   document.getElementById('cartDrawerClose').addEventListener('click', closeCartDrawer);
83 }

```

Nota. document.createElement, classList, setAttribute, innerHTML, appendChild — manipulación del DOM pura, sin frameworks. Funcionalidad 6.

Figura 48

RenderCartDrawer() – Render dinámico + badge + totales

```

105 function renderCartDrawer() {
106   const body = document.getElementById('cartDrawerBody');
107   const footer = document.getElementById('cartDrawerFooter');
108   if (!body || !footer) return;
109
110   const cart = getCart();
111   const totalQty = cart.reduce((sum, i) => sum + i.qty, 0);
112
113   // Badge en el ícono del navbar
114   const badge = document.getElementById('cartBadge');
115   if (badge) {
116     badge.textContent = totalQty.toString();
117     badge.classList.toggle('d-none', totalQty === 0);
118   }
119
120   if (cart.length === 0) {
121     body.innerHTML = `
122       <div class="cart-empty">
123         <p class="cart-empty-text">Su carrito está vacío</p>
124         <a href="tienda.html" class="btn-explore">EMPEZAR A COMPRAR</a>
125       </div>
126     `;
127     footer.classList.add('d-none');
128     footer.innerHTML = '';
129     return;
130   }
131
132   const { subtotal, total, missingForFreeShipping } = calcTotals(cart);
133
134   const shippingBanner = missingForFreeShipping > 0
135     ? `<div class="cart-shipping-banner">Gasta $/${missingForFreeShipping.toFixed(2)} más y obtén el envío gratuito</div>`
136     : `<div class="cart-shipping-banner">Tu pedido tiene envío gratuito</div>`;
137
138   const itemsHtml = cart.map(item => `
139     <div class="cart-item" data-id="${item.id}">
140       <div class="cart-item-img-wrap">
141         
142       </div>
143       <div class="cart-item-info">
144         <span class="cart-item-name">${item.name}</span>
145         <span class="cart-item-origin">${item.origin || ''}</span>
146         <div class="cart-item-bottom">
147           <span class="cart-item-price">$/${(item.price * item.qty).toFixed(2)}</span>
148           <div class="cart-qty-controls">
149             <button type="button" class="cart-qty-btn cart-qty-minus" data-id="${item.id}" aria-label="Restar cantidad">-</button>
150             <span class="cart-qty-value">${item.qty}</span>
151             <button type="button" class="cart-qty-btn cart-qty-plus" data-id="${item.id}" aria-label="Sumar cantidad">+</button>
152           </div>
153         </div>
154         <button type="button" class="cart-item-remove" data-id="${item.id}">Eliminar</button>
155       </div>
156     </div>
157   `).join('');
158
159   body.innerHTML = `${shippingBanner}<div class="cart-items">${itemsHtml}</div><p class="cart-shipping-note">Envío e impuestos calculados en el momento de la compra.</p>`;
160
161   footer.classList.remove('d-none');
162   footer.innerHTML = `
163     <div class="cart-totals-row">
164       <span>Subtotal</span>
165       <span>$/${subtotal.toFixed(2)}</span>
166     </div>
167     <div class="cart-totals-row cart-total-main">
168       <span>TOTAL :</span>
169       <span>$/${total.toFixed(2)}</span>
170     </div>
171     <a href="checkout.html" class="cart-checkout-btn">PAGAR</a>
172   `;
173 }

```

Nota. Actualiza el badge del navbar (L103–108), genera HTML de cada ítem con `map().join()` (L128–147), calcula y muestra subtotal/total formateados (L152–162). Funcionalidades 7–8.

Figura 49

DOMContentLoaded + delegación de eventos

```

176 document.addEventListener('DOMContentLoaded', () => {
177     injectCartDrawer();
178     renderCartDrawer();
179
180     const cartToggle = document.getElementById('cartToggle');
181     if (cartToggle) {
182         cartToggle.addEventListener('click', (e) => {
183             e.preventDefault();
184             openCartDrawer();
185         });
186     }
187
188     // Botones "Añadir al carrito" en las tarjetas de producto (index.html, tienda.html)
189     document.addEventListener('click', (e) => {
190         const btn = e.target.closest('.add-to-cart-btn');
191         if (!btn) return;
192
193         const card = btn.closest('[data-id]');
194         if (!card) return;
195
196         const name = card.querySelector('.product-name, .shop-name')?.textContent.trim() || 'Producto';
197         const priceText = card.querySelector('.product-price, .shop-price')?.textContent.trim() || '0';
198         const price = parseFloat(priceText.replace(/[^d.]/g, '')) || 0;
199         const img = card.querySelector('img')?.getAttribute('src') || '';
200         const origin = card.querySelector('.product-origin, .shop-origin')?.textContent.trim() || '';
201         const category = card.dataset.category || '';
202
203         addToCart({ id: card.dataset.id, name, price, img, origin, category });
204     });
205 });
206

```

Nota. Inicialización del drawer (L177–178), listener en #cartToggle (L180–186), document.addEventListener('click') con e.target.closest() para detectar botones "Añadir" en cualquier product card (L189–204). Funcionalidad 9.

Figura 50

Validacion completa de 5 campos + fetch POST al backend

```

71 // Envio del formulario
72 if (checkoutForm) {
73     checkoutForm.addEventListener('submit', async (event) => {
74         event.preventDefault();
75         const fields = {
76             ckEmail: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
77             ckNombre: /.{3,}/,
78             ckDni: /^\d{8}$/,
79             ckDireccion: /.{5,}/,
80             ckCiudad: /.{2,}/
81         };
82         let isValid = true;
83         Object.keys(fields).forEach(id => {
84             const el = document.getElementById(id);
85             if (!el) return;
86             el.classList.remove('is-invalid');
87             if (!fields[id].test(el.value.trim())) {
88                 el.classList.add('is-invalid');
89                 isValid = false;
90             }
91         });
92         if (!isValid) return;
93         const currentCart = getCart();
94         const { total } = calcTotals(currentCart);
95         const submitBtn = document.getElementById('checkoutSubmitBtn');
96         const originalText = submitBtn.textContent;
97         submitBtn.textContent = 'Procesando...';
98         submitBtn.disabled = true;
99         try {
100             const response = await fetch('/api/pedido', {
101                 method: 'POST',
102                 headers: { 'Content-Type': 'application/json' },
103                 body: JSON.stringify({
104                     cliente: {
105                         email: document.getElementById('ckEmail').value.trim(),
106                         nombre: document.getElementById('ckNombre').value.trim(),
107                         dni: document.getElementById('ckDni').value.trim(),
108                         direccion: document.getElementById('ckDireccion').value.trim(),
109                         ciudad: document.getElementById('ckCiudad').value.trim(),
110                         codigoPostal: document.getElementById('ckCodigoPostal').value.trim()
111                     },
112                     items: currentCart,
113                     total
114                 })
115             });
116             const data = await response.json();
117             if (data.success) {
118                 saveCart([]);
119                 window.location.href = 'checkout-confirmacion.html';
120             } else {
121                 alert(data.message || 'No se pudo procesar el pedido.');
```

Nota. Objeto fields con regex para email, DNI 8 dígitos y dirección mínima (L77–82), loop de validación con classList.add('is-invalid') (L85–93), fetch a /api/pedido + vaciar carrito + redirect (L106–127). Funcionalidad 10+.

Figura 51

Flujo completo: validación email, código demo, éxito

```

8 document.addEventListener('DOMContentLoaded', () => {
9   const step1 = document.getElementById('step1');
10  const step2 = document.getElementById('step2');
11  const step3 = document.getElementById('step3');
12  const emailForm = document.getElementById('emailForm');
13  const codeForm = document.getElementById('codeForm');
14  const changeEmailBtn = document.getElementById('changeEmailBtn');
15  let generatedCode = null;
16  let enteredEmail = '';
17  function showStep(step) {
18    [step1, step2, step3].forEach(s => s.classList.add('d-none'));
19    step.classList.remove('d-none');
20  }
21  function generateCode() {
22    return Math.floor(100000 + Math.random() * 900000).toString();
23  }
24  emailForm.addEventListener('submit', async (event) => {
25    event.preventDefault();
26    const emailInput = document.getElementById('regEmail');
27    emailInput.classList.remove('is-invalid');
28    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
29    if (!emailPattern.test(emailInput.value.trim())) {
30      emailInput.classList.add('is-invalid');
31      return;
32    }
33    enteredEmail = emailInput.value.trim();
34    const newsletter = document.getElementById('regNewsletter').checked;
35    try {
36      await fetch('/api/registro', {
37        method: 'POST',
38        headers: { 'Content-Type': 'application/json' },
39        body: JSON.stringify({ email: enteredEmail, newsletter })
40      });
41    } catch (error) {
42      console.error('Error de conexión:', error);
43    }
44    generatedCode = generateCode();
45    document.getElementById('sentToEmail').textContent = enteredEmail;
46    document.getElementById('demoCodeMessage').textContent = `Tu código de verificación (demo): ${generatedCode}`;
47    showStep(step2);
48  });
49  codeForm.addEventListener('submit', (event) => {
50    event.preventDefault();
51    const codeInput = document.getElementById('regCode');
52    codeInput.classList.remove('is-invalid');
53    const code = codeInput.value.trim();
54    if (!/^\d{6}$/.test(code) || code !== generatedCode) {
55      codeInput.classList.add('is-invalid');
56      return;
57    }
58    showStep(step3);
59  });
60  changeEmailBtn.addEventListener('click', () => {
61    showStep(step1);

```

Nota. generateCode() (L26–28), validación regex email (L36–40), fetch POST a /api/registro (L46–53), generación y display del código (L55–57), validación del código de 6 dígitos (L68–70). Todo modular con funciones puras.

Figura 52

Filtrado dinámico por 3 criterios

```

28 // ===== Filtro de Categorías + Búsqueda por nombre =====
29 const tagButtons = document.querySelectorAll('.tag-btn');
30 const shopItems = document.querySelectorAll('.shop-item');
31 const shopSearchInput = document.getElementById('shopSearchInput');
32
33 const applyFilters = () => {
34   const activeTag = document.querySelector('.tag-btn.tag-active');
35   const filterValue = activeTag ? activeTag.getAttribute('data-filter') : 'all';
36   const maxPrice = parseInt(document.getElementById('priceRange').value, 10);
37   const searchTerm = (shopSearchInput?.value || '').toLowerCase().trim();
38
39   shopItems.forEach(item => {
40     const itemCat = item.getAttribute('data-category');
41     const itemPrice = parseInt(item.getAttribute('data-price'), 10);
42     const itemName = (item.querySelector('.shop-name')?.textContent || '').toLowerCase();
43
44     const matchesCat = (filterValue === 'all' || itemCat === filterValue);
45     const matchesPrice = itemPrice <= maxPrice;
46     const matchesSearch = (searchTerm === '' || itemName.includes(searchTerm));
47
48     if (matchesCat && matchesPrice && matchesSearch) {
49       item.classList.remove('hidden');
50     } else {
51       item.classList.add('hidden');
52     }
53   });
54 };
55
56 tagButtons.forEach(btn => {
57   btn.addEventListener('click', (e) => {
58     tagButtons.forEach(b => b.classList.remove('tag-active'));
59     e.target.classList.add('tag-active');
60     applyFilters();
61   });
62 });
63
64 if (shopSearchInput) {
65   // Si llegamos desde la barra de búsqueda del navbar (tienda.html?q=termino), precargar el término
66   const urlParams = new URLSearchParams(window.location.search);
67   const queryTerm = urlParams.get('q');
68   if (queryTerm) {
69     shopSearchInput.value = queryTerm;
70   }
71
72   shopSearchInput.addEventListener('keyup', applyFilters);
73   applyFilters();

```

Nota. Filtra simultáneamente por categoría, precio máximo (slider) y término de búsqueda (L33–53). Lee URLSearchParams para precargarse desde el navbar de búsqueda (L66–70). Sin recarga de página.

Figura 53

IIFE de tema + toggle dark/light persistido en localStorage

```

1  /*
2   * theme.js - MC BACKPACK
3   * Alterna entre modo oscuro (por defecto) y modo claro, persistido en localStorage.
4   * Se aplica como atributo data-theme en <html> para que todo el CSS reaccione.
5   * =====
6   */
7
8  (function () {
9      const THEME_KEY = 'mc_theme';
10     const savedTheme = localStorage.getItem(THEME_KEY) || 'dark';
11     document.documentElement.setAttribute('data-theme', savedTheme);
12 })();
13
14 document.addEventListener('DOMContentLoaded', () => {
15     const THEME_KEY = 'mc_theme';
16     const toggleBtn = document.getElementById('themeToggle');
17
18     if (!toggleBtn) return;
19
20     function updateLabel() {
21         const current = document.documentElement.getAttribute('data-theme');
22         toggleBtn.setAttribute('aria-label', current === 'light' ? 'Cambiar a modo oscuro' : 'Cambiar a modo claro');
23     }
24
25     updateLabel();
26
27     toggleBtn.addEventListener('click', () => {
28         const current = document.documentElement.getAttribute('data-theme');
29         const next = current === 'light' ? 'dark' : 'light';
30         document.documentElement.setAttribute('data-theme', next);
31         localStorage.setItem(THEME_KEY, next);
32         updateLabel();
33     });
34 });

```

Nota. IIFE (L8–12) aplica el tema guardado *antes* de DOMContentLoaded para evitar destello visual. Toggle en L27–33 alterna data-theme y persiste en localStorage. Módulo autocontenido.

ANEXO C3: BACK-END CON NODE.JS Y EXPRESS.JS

Figura 54

Configuración express: middlewares + static + rutas + listen

```
1  const express = require('express');
2  const cors = require('cors');
3  const path = require('path');
4  const apiRoutes = require('./routes/api');
5
6  const app = express();
7  const PORT = process.env.PORT || 3000;
8
9  // Middleware
10 app.use(cors());
11 app.use(express.json());
12 app.use(express.urlencoded({ extended: true }));
13
14 // Serve static frontend files
15 app.use(express.static(path.join(__dirname, 'frontend')));
16
17 // API Routes
18 app.use('/api', apiRoutes);
19
20 // Fallback to index.html for frontend routing (if needed)
21 app.use((req, res) => {
22   res.sendFile(path.join(__dirname, 'frontend', 'index.html'));
23 });
24
25 // Start server
26 app.listen(PORT, () => {
27   console.log(`Servidor iniciado en http://localhost:\${PORT}`);
28 });
29
```

Nota. cors() (L10), express.json() (L11), express.static() para el frontend (L15), montaje de /api (L18), app.listen(PORT) (L26). Todo en 29 líneas — servidor minimalista y organizado.

Figura 55

3 rutas POST centralizadas en un solo router

```
1  const express = require('express');
2  const router = express.Router();
3  const contactController = require('../controllers/contactController');
4  const userController = require('../controllers/userController');
5  const pedidoController = require('../controllers/pedidoController');
6
7  // Ruta para recibir los datos del formulario de contacto
8  router.post('/contacto', contactController.saveContact);
9
10 // Ruta para registrar un nuevo usuario
11 router.post('/registro', userController.registerUser);
12
13 // Ruta para registrar un pedido del checkout
14 router.post('/pedido', pedidoController.savePedido);
15
16 module.exports = router;
```

Nota. POST /contacto, POST /registro, POST /pedido — cada una apuntando a su controller. Centralización de endpoints en una sola capa de routing, como exige la rúbrica.

Figura 56

Validación + llamada al servicio + respuestas HTTP

```

1  const contactService = require('../services/contactService');
2
3  const saveContact = async (req, res) => {
4    try {
5      const { nombre, email, mensaje } = req.body;
6
7      // Validación básica a nivel de controlador
8      if (!nombre || !email || !mensaje) {
9        return res.status(400).json({ success: false, message: 'Todos los campos son requeridos.' });
10     }
11
12     const result = await contactService.processContactData({ nombre, email, mensaje });
13
14     if (result.success) {
15       return res.status(201).json({ success: true, message: 'Mensaje enviado correctamente.' });
16     } else {
17       return res.status(500).json({ success: false, message: 'Error procesando el contacto.' });
18     }
19   } catch (error) {
20     console.error('Error in contactController:', error);
21     return res.status(500).json({ success: false, message: 'Error interno del servidor.' });
22   }
23 };
24
25 module.exports = {
26   saveContact
27 };

```

Nota. Valida nombre, email, mensaje (L7–9), llama a contactService (L12), retorna 201 / 400 / 500 según resultado (L14–22). Patrón Controller → Service → Repository documentado en el informe.

Figura 57

Registro de usuario – mismo patrón limpio

```
1  const userService = require('../services/userService');
2
3  const registerUser = async (req, res) => {
4    try {
5      const { email, newsletter } = req.body;
6
7      if (!email) {
8        return res.status(400).json({ success: false, message: 'El correo electrónico es requerido.' });
9      }
10
11      const result = await userService.processRegistration({ email, newsletter: !!newsletter });
12
13      if (result.success) {
14        return res.status(201).json({ success: true, message: 'Registro iniciado correctamente.' });
15      } else {
16        return res.status(500).json({ success: false, message: 'Error procesando el registro.' });
17      }
18    } catch (error) {
19      console.error('Error in userController:', error);
20      return res.status(500).json({ success: false, message: 'Error interno del servidor.' });
21    }
22  };
23
24  module.exports = {
25    registerUser
26  };
```

Nota. Replica el patrón: valida email (L7–9), delega a `userService.processRegistration()` (L11), responde 201 con mensaje descriptivo. Demuestra consistencia arquitectural entre los 3 controladores.

Figura 58

Pedido – valida array de items + delega a servicio

```
1  const pedidoService = require('../services/pedidoService');
2
3  const savePedido = async (req, res) => {
4    try {
5      const { cliente, items, total } = req.body;
6
7      if (!cliente || !items || !Array.isArray(items) || items.length === 0) {
8        return res.status(400).json({ success: false, message: 'Datos del pedido incompletos.' });
9      }
10
11      const result = await pedidoService.processPedido({ cliente, items, total });
12
13      if (result.success) {
14        return res.status(201).json({ success: true, message: 'Pedido registrado correctamente.' });
15      } else {
16        return res.status(500).json({ success: false, message: 'Error procesando el pedido.' });
17      }
18    } catch (error) {
19      console.error('Error in pedidoController:', error);
20      return res.status(500).json({ success: false, message: 'Error interno del servidor.' });
21    }
22  };
23
24  module.exports = {
25    savePedido
26  };
27
```

Nota. Valida que items sea un array no vacío (L7–9) — caso más complejo que los otros dos controllers. Muestra manejo de validación de estructuras de datos complejas (carrito completo).

Figura 59

Dependencias del backend – express + cors

```
1  {  
2    "name": "src",  
3    "version": "1.0.0",  
4    "description": "",  
5    "main": "index.js",  
6    "scripts": {  
7      "test": "echo \"Error: no test specified\" && exit 1"  
8    },  
9    "keywords": [],  
10   "author": "",  
11   "license": "ISC",  
12   "type": "commonjs",  
13   "dependencies": {  
14     "cors": "^2.8.6",  
15     "express": "^5.2.1"  
16   }  
17 }  
18
```

Nota. Archivo en src/package.json (no la raíz). Muestra "dependencies": {"cors":
"^2.8.6", "express": "^5.2.1"} — evidencia del entorno Node.js correctamente configurado.